Rank, Instruct, and Crack: Learning to Rank for Password Guessing

Anonymous submission

***Abstract*—Password security is significantly influenced by awareness of the tactics that potential adversaries might use. To this end, modeling guessing attacks has become a pivotal task. In pursuit of maximized attack efficacy, adversaries $\mathcal{A}$ usually attempt to guess in descending order of likelihood, akin to the way generative retrieval learning-to-rank (LTR) functions in a recommendation system, which prioritizes information for recommendation to targeted users based on predicted relevance.**

In this paper, we propose a password guessing framework based on learning to rank, named RANKGUESS. We regard passwords as sequential decision trajectories. The process of generating passwords through adversaries can be likened to a sequential decision-making framework aimed at generating highly ranked passwords. In this context, the adversaries $\mathcal{A}$ are assumed to train an agent, where the current state is represented by the password sequence generated up to that point, the action taken is to generate the next token, and the evaluation score assigned by the ranker model serves as the reward signal received. Consequently, the problem of password guessing is formulated as a Markov Decision Process (MDP) and tackled using learning-to-rank techniques. Due to the generality of our approach, RANKGUESS can be applicable to further guessing scenarios (*i.e.*, trawling guessing, targeted guessing based on personally identifiable information (PII), and conditional guessing).

By employing 12 large-scale password and six PII datasets, we demonstrate that our models are effective: (1) RANKGUESS surpasses all current state-of-the-art models and outperforms GAN-based methods by 26.29%∼43.69% (on average 34.80%); (2) When the victim's PII at site A (namely PII$_A$) is known, RANKGUESS-PII for targeted password guessing based on PII$_A$, which guesses 58.21%∼91.95% of common users within 10^{12} guesses, outperforming foremost counterpart by 6.32%∼17.09%; (3) Within 10^7 guesses, our RANKGUESS-MASK based on victims' partial passwords (*e.g.*, d**l*02*), enhances the password cracking success rate by approximately 7.70% to 14.85% (on average 8.21%) compared to state-of-the-art models. Furthermore, we have meticulously analyzed the scenarios in which our model is applicable, as well as the sources of performance improvement.**

I. INTRODUCTION

Passwords are prevalent means of user authentication [1], [9], [92] and are crucial for the online daily activities of the approximately 5.2 billion netizens [8]. With the proliferation of online services, the number of web accounts that users need to manage constantly increases, ranging from 80 to 107 [65]. However, users typically can only remember 5-7 different passwords [37]. Unavoidably, users tend to choose popular strings [80], employ personally identifiable information (PII) [81], and are susceptible to side-channel attacks [62], [90] (shoulder surfing [19], keystroke audio feedback [21], [42], *etc.*). Recently we seem to get accustomed to catastrophic password data breaches from high-profile sites (*e.g.*, 3 billion Yahoo leak [2], 1.1 billion Alibaba credential breach [57]). This sudden upsurge in account usage also exposes users' passwords to a multitude of potential threats. Thus, it is of great importance to understand password security in-the-wild.

To this end, modeling password guessing attacks has become a pivotal undertaking. In practice, guess number [61], [73] is found to be a good metric to evaluate password strength, and those easily guessed by adversaries $\mathcal{A}$ are considered to be weak passwords. Therefore, studying password strength from the adversaries' perspective is doubly profound. Moreover, the adversaries $\mathcal{A}$ may gather extra information (*e.g.*, PII [81], or partial passwords [62]) with breached passwords to guess victims' credentials and steal property/engage in deception. The incessant escalation of attacks encountered by users underscores the urgency of investigating the ever-evolving tactics employed by adversaries $\mathcal{A}$. As such, the study of continuously advancing adversaries $\mathcal{A}$ becomes imperative.

Overview of password guessing. At IEEE S&P'09, Weir *et al.* [85] proposed the first probabilistic guessing model utilizing a probabilistic context-free grammar (PCFG) [36]. Since then, a series of trawling guessing approaches that employ users' behavior of choosing popular passwords have been proposed, such as Markov-based [51], [56], LSTM-based [53], GAN-based [31], [62], and Random forest-based [84]. Moreover, there is an increase in spell-jacking [4] and credential leaks with PII (*e.g.*, 0.7 billion LinkedIn leaks with PII [3]). Furthermore, in 2015, Wang *et al.* identified these vulnerabilities and subsequently developed a range of targeted guessing models (*e.g.*, TarMarkov [79], TarGuess-I [81] and RFGuess-PII [84]). Moreover, in the case adversaries $\mathcal{A}$ obtain partial passwords (*e.g.*, `d⋆⋆⋆⋆l⋆02⋆`), Pasquini *et al.* [62] proposed the first conditional guessing model, CWAE to fill the pivots which pre-collected by adversaries. PassBERT at USENIX SEC'23 [90] uses Bidirectional Encoder Representations from Transformers (BERTs) [20] with the pre-train and finetuning paradigm.

Expert password guessing. An expertised adversaries $\mathcal{A}$ sknows how to *rank* password candidates. They know which passwords have higher priority and which have lower priority. Then, they *instruct* password guessing tools/agents to generate a predetermined list, and carry out guessing attacks to crack those passwords that match. Weak passwords that are susceptible to attacks often exhibit common features, leading to their ranking towards the top. This is a classic and effective paradigm of password guessing attack. However, this requires a vast amount of expert knowledge to decide which is valuable. To this end, we can train a guessing agent to *instruct* (replace typical ones) guess candidates generation to assist in guiding the guessing of high-value passwords (*i.e.*, weak ones).

Network and Distributed System Security (NDSS) Symposium 2025
23 February - 28 February 2025, San Diego, CA, USA
ISBN 1-891562-83-5
https://dx.doi.org/10.14722/ndss.2025.25xxx
www.ndss-symposium.org

A. Motivation and Challenges

Learning-to-rank (LTR) [22], [55], [75] plays a crucial role in generative retrieval [48] systems: (1) Machine translation: ranking the translations of a set of hypotheses. (2) Computational biology: ranking the candidate 3-D structures in protein structure prediction. (3) Recommendation systems: ranking relevant news articles for recommending to users after they have read the current news article. Reward models are widely used to assess sequence generation in large language modelings (*e.g.*, RLHF [52], [68], using given prompts to get the reward from human evaluation), to align with human preferences as much as possible. Password guessing still has a clear preference as Equation 1. Motivated by the above analysis, a natural research question arises: *is it possible to propose a password guessing agent (i.e., guesser) rivals as a guessing expert, to align the preferences of learning to rank in password guessing*. To answer the question is not straightforward, and here we explain why.

In *ad-hoc* password guessing task, given an additional *query* information $q \in \mathcal{Q}$ (*i.e.*, PII, and partial password) and a set of N password candidates $\mathcal{G} = \{p_1, p_2, \ldots, p_N\}$ from a password space $\mathcal{C}$, a guessing model f aims to associate a relevance score $f(q, p_n)$ with each pair $(q, p_n) \in \mathcal{Q} \times \mathcal{C}$ to rank the whole password set. For example, the ranking model outputs the ranked list $L = [p_N, p_{N-1}, \ldots, p_1]$ if it determines that

$$f(q, p_N) > f(q, p_{N-1}) > \cdots > f(q, p_1), \tag{1}$$

as adversaries $\mathcal{A}$ usually attempt to guess in descending order of likelihood, just as generative retrieval learning to rank. We aim to establish a password guessing model like the one described in Section I, capable of interacting with the environment, perceiving the strength of passwords, and thereby generating guesses. We divide the process into three parts:

- Learning to generate: generate guess candidates for different scenarios#1-#3 in Table I via the guesser,
- Learning to rank: learn which passwords are ranking towards the top in password space $\mathcal{C}$ via the ranker.
- Instruct to guess: with the aforementioned two capabilities, the ranker can instruct the guesser to guess.

More challenges emerge as the above analysis surfaces:

Ch1: Even minor errors would result in the failure of password guesses. Passwords are extremely short texts with rich semantics and syntax. In password guessing, unlike traditional generative retrieval [48], small changes in passwords can lead to guessing failure. For example, if a model generates `daniel1027` it would fail at guessing `d@niel1027`. This imposes more constraints on accurate matching above Section I-A mentioned three tasks [38], [84] and needs more additional restrictions when generating guess candidates.

Ch2: How to design rewards for password guessing. Password guessing can be regarded as a learning-to-rank problem, which generates guesses ranking via likelihood in descending order. Fundamentally, a reward signal is an abstraction of the task and serves as the bridge through which we convey the task objective to the agent. Designing rewards in password guessing can be a daunting task, as defining desired behaviors through reward signals can be challenging or may necessitate numerous expert demonstrations. During the parts of learning to generate/rank, we need to assign a reward to each state of the password, followed by the calculation of the cumulative reward collection for the entire generation trajectory.

Ch3: How to utilize additional information to guess users' passwords. The construction of passwords typically involves PII or popular texts, and adversaries $\mathcal{A}$ may acquire partial passwords (*e.g.*, `d****l*02*`) through various means. How can a guessing model for individual users be learned from the additional available information? Wang *et al.* proposed solutions for traditional matching methods in Targuess-I [81] and RFGuess-PII [84]. However, for deep learning approaches, there is no clear-cut solution here. The challenge is to achieve the highest success rate by utilizing additional information in password guessing with a limited number of attempts.

B. Our contributions

In this work, our key contributions are three-fold:

- **A new technical password guessing route.** We build a password guessing model using learning to rank technique, treating passwords as trajectories. We, *for the first time*, define the learning to rank rewards for the password guessing task. Then, We employ ranker-instructed rewards at each timestep t during guessing Monte Carlo rollouts, facilitating policy gradient optimization. It learns from the leaked password dataset to generate high-quality guess candidates.
- **Three improved password guessing models.** We systematically analyze various experimental results. Our RankGuess demonstrates exceptional performance than state-of-the-art (SOTA) in trawling guessing by 2.83% $\sim$ 10.91% (avg. 6.91%) and outperformed GAN-based SOTAs by 26.29% $\sim$43.69% (avg. 34.80%). RankGuess-PII, when armed with the victim's Personally Identifiable Information (PII_A) at site A, achieves targeted guessing with a remarkable success rate of 58.21% to 91.95% within 10^{12} guesses, outperforming SOTA by 6.32% to 17.09%. RankGuess-Mask, based on victims' partial passwords (*e.g.*, `d****l*02*`), significantly enhances the cracking success rate by 7.70% to 14.85% (avg. 8.21%) within 10^7 guesses, surpassing latest SOTAs.
- **Some insights.** When it comes to predicting the next action in the state *n*-gram patterns, RankGuess employs a reward model via learning to rank, which converts non-differentiable evaluation function scores and proceeds with policy gradient updates. RankGuess can better detect PII in Chinese passwords, and sequential information in English passwords, and has enhanced capabilities for capturing substrings (len$\geq$6). We also designed a corresponding password strength meter (PSM) based on the model to protect users and proposed and conducted measure accuracy and unsafe error evaluations in passwords.

II. Background and Related Work

A. Three guessing scenarios

1) Trawling password guessing: Trawling guessing means that the adversaries $\mathcal{A}$ does not care who the specific target is, and the only goal is to guess more passwords under the

TABLE I. SUMMARY OF THREE PASSWORD GUESSING SCENARIOS WITH TOY EXAMPLES, LABELED SETS AND CORRESPONDING EXPLANATIONS.

Attack scenarios	**Toy example** $(x \rightarrow y)$ with $(x \in X, y \in Y)$	**Labeled sets** Inputs X, accompanied by its outputs Y	**Explanations of attack**
Trawling guessing	B_s `daniel1027` $\rightarrow$ `daniel1027` E_s	Passwords with the begin symbol B_s with them shifted right with the end symbol E_s.	The attacker does not care who the specific target is, and its only goal is to guess as more passwords as it is possible [78].
Targeted guessing	(`daniel@xxx.com`, `daniel`, `lavender`, `10271985`, `123-456-7890`)$\rightarrow$`daniel1027`	PII (email, user name, name, birthday, phone, *etc.*) with corresponding targeted passwords.	Utilizing models to inference password with provided specific users' PII [81].
Conditional guessing	`d****l*02*` $\rightarrow$ `daniel1027`	Partial passwords with complete passwords.	Complete password by filling the pivot [62].

guess number allowed (*e.g.*, $> 10^9$ as recommended by Wang *et al.* [78]). At IEEE S&P'09, Weir *et al.* [85] proposed the first password guessing models based on probabilistic context-free grammar (PCFG) [36]. In 2014, Ma *et al.* [51] implied that the PCFG algorithm is statistics-based models that crack passwords by counting the frequency of elements in the training set (e.g., the LDS segments in PCFG), which have the inherent limitations of password sparseness and overfitting [51]. Meanwhile, Narayanan *et al.* [56] introduced the Markov chain [35], [67] to capture dependencies between passwords. In 2019, Hitaj *et al.* [31] proposed PassGAN, a method that learns password samples between the real and generated passwords to simulate password distribution. They use a generative neural network to capture password features and a discriminative neural network [28] to assess the similarity between training and generated ones. At this point, it indicates that the generator has the capability to generate real passwords. After that, Pasquini *et al.* [62] alleviated the mode collapse problem of training GANs and Wasserstein autoencoder via WGAN and CWAE [1], improving the cracking rate. On this basis, they constructed two guessing frameworks, that is, Conditional Password Guessing (CPG) and Dynamic Password Guessing (DPG). However, both PassGAN [31] and Pasquini *et al.* [62] are not probabilistic models, which simply utilize ResNet [30] as their generator and discriminator without accounting for the cyclic dependencies within passwords.

2) Targeted password guessing: There are many kinds of PII about users, such as demographic related information (name, birthday, age, phone number, education, gender, *etc.*). The goal of targeted password guessing is to crack the password of a given user in a given targeted PII as quickly as possible [81]. In 2015, Wang *et al.* [79] introduced the first targeted probabilistic model known as TarMarkov. Grounded in the principles of Markov [56], this approach was designed to leverage the frequency of names within a training password set in order to estimate the likelihood of specific users choosing name-based passwords. In 2016, Li *et al.* [44] proposed a targeted guessing model based on PCFG [85]. After that, Wang *et al.* [81] showed that their TarGuess-I is more efficient than models using PII to crack passwords, which can gain success rates over 20% with 100 guesses. TarGuess-I adds six PII tags (name, username, birthday, phone number, *etc.*) to the three basic tags LDS segments in the PCFGs guessing algorithm [36].

3) Conditional password guessing: Conditional password guessing means that an attacker $\mathcal{A}$ has obtained an incomplete password (*e.g.*, some wildcards) of the victim by exploiting various side channel attacks [19], [21], [42], [62], [90] (*e.g.*, shoulder surfing, keystroke audio feedback, and thermal residue), then initiates a password guessing attack (*e.g.*, `e***a*rt*`, `d****l*02*`). A plain way to tackle conditional password guessing is by using trawling models to create numerous potential passwords [62]. Then, these potential passwords could be filled into the wildcard through a filtering process to get guess. However, the task of efficiently removing irrelevant passwords from this pool can be resource-intensive and demand significant storage capacity.

At IEEE S&P'21, Pasquini *et al.* [62] proposed the first deep conditional password guessing model based on Wasserstein Autoencoder [72]. They refer to the final model as context wasserstein autoencoder (CWAE) which consists of an encoder embedding a partial password into latent representations, and a decoder converting the latent representations of the partial password into passwords. In 2023, Xu *et al.* [90] proposed a state of the art approach for conditional password guessing based on bi-directional encoder representation from transformers (BERT) [20] using pre-train and finetuing paradigm [49]. However, PassBERT [90] requires a massive pretraining password to achieve results competitive with CWAE [62], which itself represents a biased advantage over other models. Additionally, BERT [20] was originally designed to address issues of long-range dependencies and parallel processing between words, which are not pertinent concerns in password guessing. Therefore, choosing Transformer [77] models like BERT family [20] as the backbone network may not be a suitable option network for password guessing.

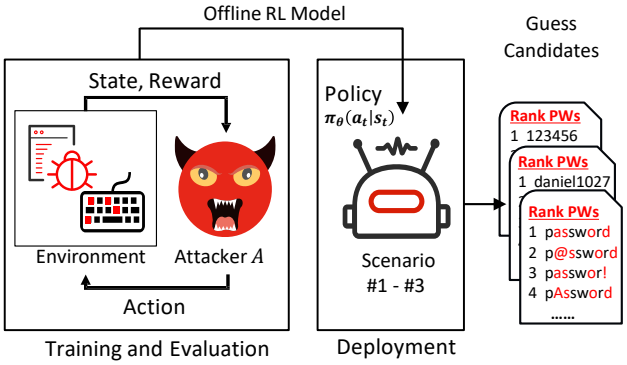


Fig. 1. An example of the offline RL models for three password guessing attacks trawling, targeted and conditional scenarios#1-#3 as Table I. It makes decisions about the best next actions to maximize its returns based on the current state by perceiving the rewards from the environment.

Summary. We summarized the training sets, denoted as X, and the generated guesses Y for the three password guessing scenarios in Table I. Moreover, we provided an overview of the information advantage held by each adversaries $\mathcal{A}$. In the trawling scenario, there is no additional information available to the adversaries $\mathcal{A}$. In the targeted password scenario, the attacker possesses the victim's password using PII. In the conditional password scenario, the attacker has access to the victim's partial passwords leaked by some side-channel means.

[1]More specifically, Pasquini *et al.* [62] maintain the core GANs and Autoencoders framework and obtain exceptional performance via Wasserstein GAN and Context Wasserstein Autoencoder, abbr. as WGAN and CWAE.

B. Reinforcement learning

Previous models (*e.g.*, Statistics-based [51], [85], Random forest-based [84] and FLA-based [53]) included both input and corresponding expected output. These models are learned by minimizing the difference between the predicted output and the actual labels. They only learn from password data with the correct labels. However, in reinforcement learning (RL) [70], the model needs to be explored during the training, trying out new actions to discover better policies than traditional ones.

Reinforcement Learning [70] is a machine learning approach where agents learn to make decisions by balancing exploration with exploitation. It's been applied to games, natural language processing (NLP), and information retrieval. Recently, there's been a growing trend in using RL methods to generate natural language for NLP tasks [45], [91]. In RL, an agent completes an episode by producing an end symbol and receives a reward based on the action quality of generated tokens. The agent's policy, $\pi_\theta(a_t|s_t)$, gives a probability for each action a_t in state s_t, with θ as the policy parameters. The agent explores by sampling from this action distribution rather than always choosing the most likely action. To summarize, in password guessing, the agent aims to find the best policy π^* with optimal parameters θ^* to maximize its guessing performance.

Learning to rank. Password guessing is a learning to rank process that consistently prioritizes high-likelihood passwords for attacks, as explained in Section II-A. We can consider this process as an expert strategy $\mathcal{E}$, and our goal is to train an agent capable of acquiring the skill of password guessing. This naturally constitutes an imitation learning problem [29], [40], [58], [66], [69], which is a more effective strategy for a sequential generation with near-optimal cost-to-go oracles [69]. The essence of generative adversarial imitation learning (GAIL) is to imitate the strategy of the expert $\mathcal{E}$ in the environment. We consider the interaction with the expert $\mathcal{E}$ to signals when training differently with Jonathan *el al.* [33].

III. Setting and RL formulation

A. Password guessing modeling

Suppose the password token set in this work is denoted by $\mathcal{V}$, and at timestep t, the previous tokens generated in the sequence are $\text{PW}_{1:t} = (w_0, w_1, ..., w_{t-1})$ as Figure 1, where all tokens $w_i \in \mathcal{V}$ in Table IX and Table VIII. To formalize this problem, we regard password generation $\text{PW}_{1:T} = (w_0, w_1, ..., w_{T-1})$ as a sequential decision process [64]:

$$\tau = \{s_0, a_0, s_1, a_1, \ldots, s_T, a_T\}. \tag{2}$$

At timestep t, action a_t could be considered as to choose a token from the token set $\mathcal{V}$, with the policy denoted as $\pi_\theta(a_t \mid s_t)$, where the state s_t is the t-gram in previous steps $\text{PW}_{1:(t-1)}$. In such a case, the offline reinforcement learning (offline RL) model aims: to learn an optimal (or nearly optimal) policy from a pre-collected password D without an interactive environment. We use $\mathbb{S}$ and $\mathbb{A}$ to represent the RL models' input and output space formally called state and action in RL scenes. $r_t \in \mathbb{R}$ is the temporal reward for each timestep t, where $\mathbb{R}$ is the real number set. A unit in a pre-collected password called transition is a four-element set: $\{s_t, a_t, r_t, s_{t+1}\}$, where $s_t \in \mathbb{S}, a_t \in \mathbb{A}$, and $s_{t+1} \in \mathbb{S}$ is the successive state of s_t. The probability of a trajectory τ is given by the format of

$$\begin{aligned} P_\theta(\tau) &= P_\theta(s_0, a_0, s_1, a_1, \cdots, s_T, a_T) \\ &= P_\theta(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P_\theta(s_{t+1} \mid s_t, a_t). \end{aligned} \tag{3}$$

For example, $\text{PW}_{1:t} = \{\texttt{P}, \texttt{a}, \texttt{s}, \texttt{s}, \texttt{w}, \texttt{@}, \texttt{r}, \texttt{d}\}$, we set s_0 as B_s, then the action a_0 is chosen to obtain the maximum reward with 0.62 for the trajectory of token $w_0 = \texttt{P}$, resulting in $s_1 = \mathsf{B}_s\texttt{P}$, then make next action a_1 to get $s_2 = \mathsf{B}_s\texttt{Pa}$, and so on, until the complete password `Passw@rd` is generated. And a set of transitions in chronological order forms a trajectory in password D. Based on the transitions, the offline RL model learns the Markov Decision Process underneath the passwords and forms a policy $\pi_\theta(a_t \mid s_t)$ such that the password ranking expected return $\mathfrak{L}(\theta)$ is maximized [32], [69]:

$$\begin{aligned} \mathfrak{L}(\theta) &= \mathbb{E}_{\tau \sim P_\theta(\tau)}[\mathcal{T}(\tau)] \\ &= \mathbb{E}_{\text{PW}_{1:T} \sim P_\theta(\text{PW}_{1:T})}[\mathcal{T}(\text{PW}_{1:T})], \end{aligned} \tag{4}$$

where $\mathcal{T}(\text{PW}_{1:T})$ represents the total reward for the password $\text{PW}_{1:T}$. This is the first attempt at modeling learning to rank for guessing in this manner, utilizing RL to train a password guessing model. Thus, we name our model as RankGuess. The goal is to maximize the return of the guesser $\mathcal{G}_\theta$.

B. Learning to rank reward function

Regarding the mentioned reward in Section III-A, we need to design an appropriate one for password guessing. To determine the strategy our expert employs for guessing, we rank the passwords in the training set to obtain the top-1w passwords or tagged tokens trajectory as our expert $\mathcal{E}$ to imitate.

Ranking scores. Taking inspiration from the ranking steps used in network search algorithms like PageRank [59], we introduce a similarity metric capable of assessing relative ranks between PWs and formulate the ranking score of PW concerning a given expert $\mathcal{E}$ and comparison $\mathcal{S}^+$ as follows:

$$P(\text{PW}|\mathcal{E}, \mathcal{S}^+) = \frac{\exp\left[\cos(\mathbf{y}_{\text{PW}}, \mathbf{y}_{\mathcal{E}})\right]}{\sum_{\text{PW}' \in \mathcal{S}^+ \bigcup \{\text{PW}\}} \exp\left[\cos(\mathbf{y}_{\text{PW}'}, \mathbf{y}_{\mathcal{E}})\right]}. \tag{5}$$

Here, $\mathbf{y}_{\text{PW}}$ and $\mathbf{y}_e$ are the feature vectors of the input and expert password. For a set, the ranking score is the expectation of each individual over the expert $\mathcal{E}$, denoted as $\mathcal{R}_\varphi(\text{PW}|\mathcal{E}, \mathcal{S}^+)$.

Monte-Carlo (MC) rollouts. Note that in traditional RL, the current reward is compromised by the rewards from intermediate states and future states. However, in password guessing, the guesser $\mathcal{G}_\theta$ obtains the reward if and only if one password has been completely generated, which means no intermediate reward is gained before the sequence hits the end symbol. To relieve this problem, we utilize the MC rollouts methods [11], [45], [54], [91] to simulate intermediate rewards:

$$\mathcal{Q}_{\theta,\varphi}(\text{PW}_{1:t-1}, \mathcal{E}) = \mathop{\mathbb{E}}_{\text{PW} \sim \mathcal{G}_\theta}\left[\mathcal{R}_\varphi(\text{PW}|\mathcal{E}, \mathcal{S}^+, \text{PW}_{1:t-1})\right], \tag{6}$$

Here, PW represents the complete password sampled by rollout methods at state s_{t-1}. To be more specific, the beginning tokens $(w_0, w_1, ..., w_{t-1})$ are fixed, and the rest tokens are consecutively sampled by $\mathcal{G}_\theta$ until the last token w_T is generated. Assuming we conduct N trials and obtain N trajectories,

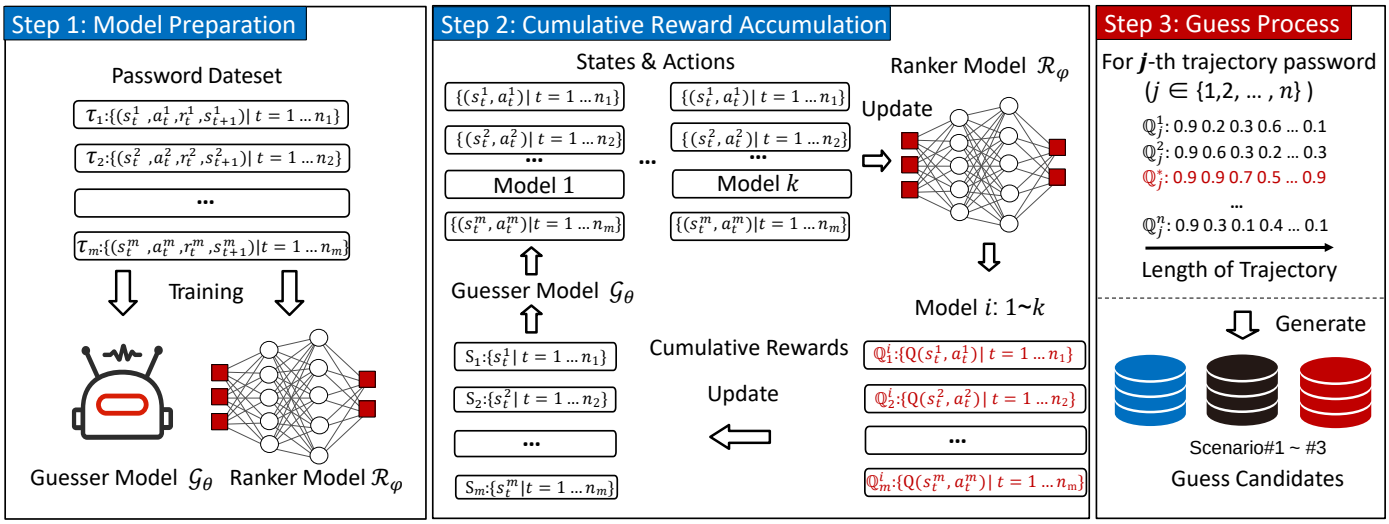


Fig. 2. The workflow of RankGuess contains three steps, *i.e.*, model preparation, cumulative reward accumulation, and guess process. RankGuess first pretrain a guesser model and a ranker model via Section III-C on the target dataset, then collects the cumulative rewards via Section III-B from the state-action pairs of password. After k iterations, finally, RankGuess can sense the reward process of every trajectory and thus be used to generate guess candidates.

denoted as $\tau^{(1)}, \tau^{(2)}, \dots, \tau^{(N)}$, with corresponding state value of $\mathcal{Q}_{\theta,\varphi}(\tau^{(i)})$, the state value of s_t can be approximated as:

$$\tilde{Q}_{\theta,\varphi}\left(\mathrm{PW}_{1:t-1}, \mathcal{E}\right) = \frac{1}{N}\sum_{n=1}^{N} \mathcal{Q}_{\theta,\varphi}(\tau^{(n)}), \tag{7}$$

As $N \to \infty$, the empirical state value function estimator $\tilde{Q}_{\theta,\varphi}\left(\mathrm{PW}_{1:t-1}, \mathcal{E}\right)$ converges in probability to the true value $\mathcal{Q}_{\theta,\varphi}(\mathrm{PW}_{1:t-1}, \mathcal{E})$ (as demonstrated in Appendix C) which denoted as δ_t for short as the empirical state value function.

C. Workflow

Step. 1: Model Preparation (MP). In the box to the left in Figure 2, the adversaries $\mathcal{A}$ prepares the guesser model $\mathcal{G}_\theta$ and the ranker model $\mathcal{R}_\varphi$ based on the target password, which contains m trajectories τ with the length of $T(i \in \{1, 2, \dots, m\})$. The ranker model is optimized to estimate the cumulative reward of each state-action pair. For each trajectory in the password, a series of predictions for its state-action pairs compose the exclusive feature for learning to rank. We use the Monte-Carlo based (MC) strategies to optimize the ranker model, *i.e.* in Section III-B. In addition, the adversaries $\mathcal{A}$ train a set of guesser models with conventional MLE methods to the supervised pre-training with model initializations. Note that the tokens for different password guessing scenarios #1∼#3 are different, as detailed in Section IV in the subsequent text.

Step. 2: Cumulative Reward Accumulation (CRA). In the middlebox, the guesser $\mathcal{G}_\theta$ model observes the states of the password and takes actions. For i-th trajectory in the password, the $\mathcal{R}_\varphi$ observes the state s_t^i and the action a_t^i of each iteration model, where a_t^i represents the guesser model's action at the t-th step of trajectory τ_i, representing the learned policies with model initialization and training processes on the password dataset. Using the learning to rank reward in Section III-B, the ranker $\mathcal{R}_\varphi$ calculates the estimations for all state-action records, *i.e.*, the estimated cumulative rewards, which are the samplings of the exact cumulative rewards. Iteratively, we update the guesser and ranker model k times.

Step. 3: Guess Process (GP). After the above two steps, the guesser $\mathcal{G}_\theta$ obtains the estimated cumulative rewards from the ranker models $\mathcal{R}_\varphi$ then conducts the guess process. For j-th trajectory, the guesser $\mathcal{G}_\theta$ knows *i.e.*, $\left\{\mathbb{Q}_j^i \mid i \in \{1, 2, \dots, k\}\right\}$ which obtain best reward, *i.e.*, $\mathbb{Q}_j^*$. We will detail the algorithms adapted for different attack scenarios in Section IV.

D. Implement detail of workflow

Due to the need for sampling during Step.2, the non-differentiability caused by sampling for discrete token generation makes gradients unable to back-propagate to the guesser $\mathcal{G}_\theta$. Therefore, we need to convert the non-differentiable loss into a reward signal using a policy gradient for model updates.

Training guesser. With the feasible intermediate rewards, we can finalize the objective function for a complete password. Refer to the proof in Appendix B, the gradient of the objective function for guesser $\mathcal{G}_\theta$ can be formulated as:

$$\nabla_\theta \mathfrak{L}_\theta = \mathop{\mathbb{E}}_{\mathrm{PW}_{1:T}\sim\mathcal{G}_\theta}\left[\sum_{t=1}^{T}\sum_{a_t\in\mathbb{A}} \nabla_\theta \delta_t \log \pi_\theta(a_t|\mathrm{PW}_{1:t-1})\right], \tag{8}$$

where ∇_θ is the partial differential operator. $\mathbb{E}_{s_{1:T}\sim\mathcal{G}_\theta}$ is the expectation over all sampled complete password based on current guesser's parameter θ within one batch. Note that we only compute the partial derivatives for θ, as the $\mathcal{R}_\varphi$ is fixed during the training of the guesser. Importantly, different from the policy gradients methods in other works [31], [62], [91], our method replaces the simple binary outputs with a ranking system, which can better reflect the quality of the imitate password and facilitate effective training of the guesser $\mathcal{G}_\theta$.

Training ranker. To train the ranker's parameters set φ, we keep the θ parameters fixed. We generate samples $\mathcal{S}^-$ using the guesser $\mathcal{G}_\theta$ and sample the same size passwords from the ground truth as $\mathcal{S}^+ \sim \mathcal{P}_h$ in $\mathcal{D}$. Next, we create mixup shadow samples $\mathcal{S}_i^M = \lambda_i \mathcal{S}^+ + (1-\lambda_i)\mathcal{S}^-$, where blending ratios $\lambda_i \sim \mathcal{U}(N)$. Then, we obtain the mean rank distribution $\mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M, \mathcal{E})$, and update ϕ via the following:

$$\nabla_\varphi \mathfrak{L}_\varphi = -\sum_{i=0}^{T-1} \nabla_\varphi \mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M, \mathcal{E}) \log \lambda_i, \tag{9}$$

by measuring the difference between the rank distribution

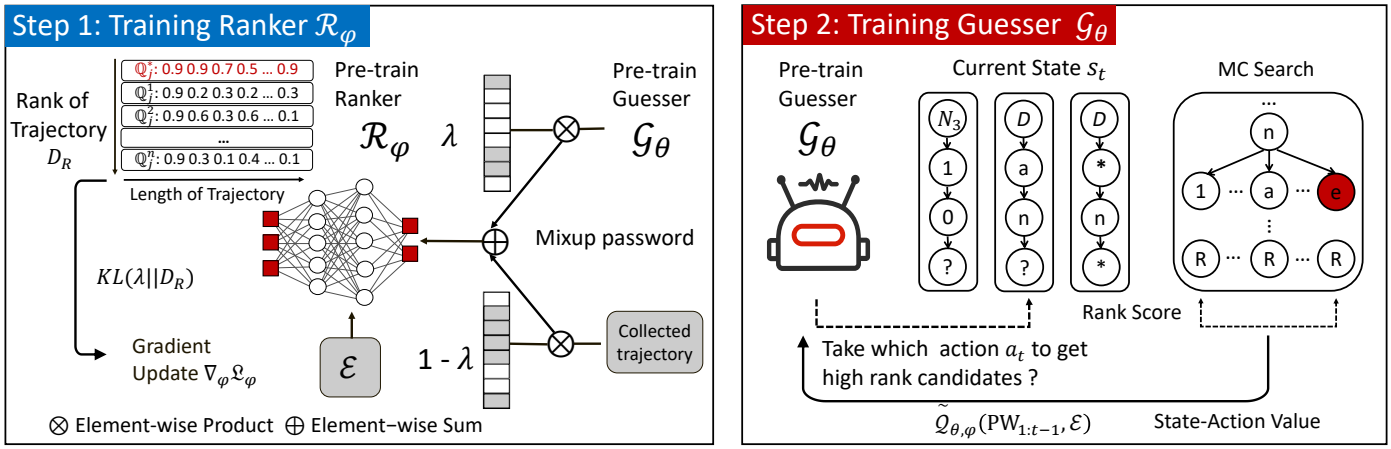


Fig. 3. Our RANKGUESS training phases are primarily divided into two steps: (1) training ranker $\mathcal{R}_\varphi$ and (2) training guesser $\mathcal{G}_\theta$. The training of ranker $\mathcal{R}_\varphi$, is to assign corresponding rankings to mixup passwords under the guidance of expert $\mathcal{E}$, which represents the Top-1w passwords from the training set serving as the expert for imitation. $\mathcal{S}^+$ consists of n randomly samples from the training set, $\mathcal{S}^-$ comprises n passwords generated by $\mathcal{G}_\theta$. By mixing these sets with blending ratios ranging from 0 to 1, we create n mixup password sets. The expected distribution is monotonically decreasing due to the changing blending ratios λ. Meanwhile, the training of the guesser $\mathcal{G}_\theta$ generates a password with a high reward passing back to the intermediate action value via Monte Carlo (MC) tree search [12] in Section III-B. Through the optimizations above, guesser can generate high-value passwords for various scenarios under the guidance of ranker.

and the true λ_i distribution using Kullback–Leibler divergence. We can find that the two parts of our training have inherent adversarial attributes. That's because the ranker has a razor-sharp sense of what's legit and what's not, while the guesser gain strong ability to generate guesses:

- The learning objective of the guesser $\mathcal{G}_\theta$, is to have the ranker $\mathcal{R}_\varphi$, assign a higher ranking rewards to the generated sample $\mathcal{S}^-$ compared to the real ones $\mathcal{S}^+$.
- The learning objective of the ranker $\mathcal{R}_\varphi$, is to assign a lower rewards to the generated sample $\mathcal{S}^-$ via the guesser model $\mathcal{G}_\theta$ compared to the real ones $\mathcal{S}^+$.

In this assumption, the ranker $\mathcal{R}_\varphi$ continuously rewards the guesser $\mathcal{G}_\theta$ under the guidance of expert $\mathcal{E}$ to enhance the guesser's generative capability. Consequently, this can be regarded as a process where $\mathcal{G}_\theta$ and $\mathcal{R}_\varphi$ engage in a minimax game. The proposed password ranker $\mathcal{R}_\varphi$, shares a similar recurrent neural network structure. Firstly, it maps the connected password matrix into embedded feature vectors $\mathbf{y}_s$ through a series of nonlinear functions $\mathfrak{F}$. Subsequently, ranking rewards for the password features $\mathbf{y}_s$ are computed using expert features $\mathbf{y}_\mathcal{E}$ extracted in advance by $\mathcal{R}_\varphi$. Firstly, we construct the ranker $\mathcal{R}_\varphi$. Its input is a piece of password, and its output is a value representing the ranking of this password as a genuine password. This output serves as the reward for the input, and it will be passed back to the generator at the end of each round for the update of policy parameters θ in the guesser. Next, we build the guesser $\mathcal{G}_\theta$, incorporating the policy function $\pi_\theta(a_t|s_t)$ mentioned earlier, where θ corresponds to the parameters of the guesser model $\mathcal{G}_\theta$. The guesser will be updated according to the previously mentioned policy gradient as Equation 8. GRU has two gates: the update gate and the reset gate. To prototype the model discussed, we use GRU [14] as the guesser model $\mathcal{G}_\theta$, which maps the input embedding representations $(\mathbf{e}_0, \ldots, \mathbf{e}_{T-1})$ of the sequence $\mathrm{PW}_{1:T} = (w_0, \ldots, w_{T-1})$ into a sequence of hidden states $(\mathbf{h}_0, \ldots, \mathbf{h}_{T-1})$ by using the update $\mathbf{g}$ recursively,

$$\mathbf{h}_t = \mathbf{g}\left(\mathbf{h}_{t-1}, \mathbf{e}_t\right). \tag{10}$$

Moreover, a softmax output layer $\mathbf{z}$ maps the hidden states into the output action state distribution taken by the guesser $\mathcal{G}_\theta$,

$$\mathbf{z}\left(\mathbf{h}_t\right) = \mathrm{softmax}\left(\mathbf{c} + \mathbf{V}\mathbf{h}_t\right), \tag{11}$$

where $\mathbf{c}$ and $\mathbf{V}$ are bias vector and weight matrix. To deal with the common vanishing and exploding gradient [27] of the backpropagation through time, we leverage the GRU cells [14] to implement the update $\mathbf{g}$ in Equation. (10) to control the sequential trajectory and flow of information in Algorithm 1:

Algorithm 1: Training RANKGUESS Framework.

Input: Password or PII tagged set $\mathcal{D}$, training epoch $\mathcal{T}$, iteration count $\mathcal{K}$, state space $\mathbb{S}$, action space $\mathbb{A}$, differentiable policy $\pi_\theta\left(a_t|s_t\right)$, learning rate α, β.
Output: Policy $\pi_\theta\left(a_t \mid s_t\right)$.

Randomly initialize parameters θ and φ;
Pe-train $\mathcal{G}_\theta$ using MLE on $\mathcal{D}$ and update θ.
Generate $\mathcal{S}^-$ samples using $\mathcal{G}_\theta$ for training $\mathcal{R}_\varphi$.
Pre-train D_ϕ via minimizing the cross entropy.
for $t \leftarrow 0$ **to** $\mathcal{T}-1$ **do**
  /* Training ranker model $\mathcal{R}_\varphi$*/
  for $k \leftarrow 0$ **to** $\mathcal{K}-1$ **do**
    Generate samples $\mathcal{S}^- \sim \mathcal{G}_\theta$ and $\mathcal{S}^+ \sim \mathcal{P}_h$ in $\mathcal{D}$;
    mixup shadow samples $\mathcal{S}_i^M = \lambda_i \mathcal{S}^+ + (1-\lambda_i)\mathcal{S}^-$, where $\lambda_i \sim U(N)$;
    Get the mean rank distribution $\mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M, \mathcal{E})$ via Equation. (6);
    $\varphi \leftarrow \varphi - \alpha \sum_{i=0}^{T-1} \nabla_\varphi \mathcal{T}_{\theta,\varphi}(\mathcal{S}_i^M, \mathcal{E}) \log \lambda_i$;
  /* Training guesser model $\mathcal{G}_\theta$*/
  Generate n trajectories $\tau^{(i)} = \left\{s_1^{(i)}, a_1^{(i)}, s_2^{(i)}, a_2^{(i)}, \ldots, s_T^{(i)}, a_T^{(i)}\right\}$ based on the strategy $\pi_\theta\left(a \mid s\right)$;
  for $l \leftarrow 0$ **to** $\mathcal{K}-1$ **do**
    $\delta \leftarrow \mathcal{T}_{\theta,\varphi}(\mathrm{PW}_{1:t-1}, \mathcal{E})$via MC search;
    $\theta \leftarrow \theta + \beta \mathop{\mathbb{E}}_{\mathrm{PW}_{1:T} \sim \mathcal{G}_\theta} \left[\sum_{t=1}^{T} \sum_{a_t \in \mathbb{A}} \nabla_\theta \delta_t \log \pi_\theta(a_t|\mathrm{PW}_{1:t-1})\right]$;
return $\pi_\theta\left(a \mid s\right)$

TABLE II. BASIC INFORMATION REGARDING OUR 12 PASSWORD DATASETS USED IN OUR EXPERIMENTS, INCLUDING DETAILS ON THE WEB SERVICE, LANGUAGE, WHEN LEAKED, UNIQUE PASSWORDS AFTER FILTERING, WITH PII OR NOT, AND THE SIZE OF THOSE BRANCH PASSWORD DATASETS.†

Dataset	Web service	Language	When leaked	Total PWs	Length>30	Removed %	Unique PWs	With PII
Taobao	E-commerce	Chinese	Feb., 2016	15,072,418	88	0.01%	11,633,759	
Dodonew	E-commerce	Chinese	Dec., 2011	16,283,140	13,4758	0.15%	10,135,260	
CSDN	Programmer	Chinese	Dec., 2011	6,428,632	0	0.01%	4,037,605	
Wishbone	Social forum	English	Jan., 2020	10,092,037	250	0.01%	5,933,902	
Mate1	Dating website	English	Mar., 2016	27,401,505	12,430	0.06%	11,916,080	
000Webhost	Web hosting	English	Oct., 2015	15,299,907	4,159	0.76%	10,526,769	
Twitter(𝕏)	Social forum	English	June,2016	40,669,963	282,149	0.69%	10,583,709	
LinkedIn	Job hunting	English	Jan., 2012	54,656,615	17,162	0.22%	34,282,741	
Rockyou	Social forum	English	Dec., 2009	32,603,387	3,140	0.07%	14,326,970	
12306	Train ticketing	Chinese	Dec., 2014	129,303	129,303	0	117,808	✓
ClixSense	Paid task platform	English	Sep., 2016	2,222,045	0	0	1,628,018	✓
Rootkit	Hacker forum	English	Feb., 2011	69,330	5	0.01%	56,835	✓

†PWs represents passwords, and PII for personally identifiable information. We filter out passwords exceeding 30 in length or contain non-ASCII characters.

IV. EXPERIMENTAL RESULTS

A. *Trawling Password Guessing* (Scenario#1)

Trawling password guessing [84], as one of the most traditional and common attack scenarios, involves attempting password guesses without using any personally identifiable information (PII) or extra information (*e.g.*, partial passwords). It is currently one of the most fundamental yet crucial guessing attack threats to password security. The trawling password guessing model can be extended to other speculative scenarios under certain constraints using additional information.

Algorithm 2: Trawling password algorithm.

Input: The threshold $\mathcal{T}$ of generated passwords.
Output: Passwords set $\mathcal{P}$.

Init LUT: look up table{key : $prefix$, value : $prob$}
$\mathcal{Q}.push(\mathsf{B}_s.tokenize())$ /* Implement a tree using a list, build a ordered search tree using BFS.*/
while *!Q.empty()* **do**
  $cur_prefix \leftarrow \mathcal{Q}.pop()$
  $next_prob \leftarrow guesser\ \mathcal{G}_\theta(cur_prefix)$
  $char_set\ \mathcal{V} \leftarrow$ top_ω(cur_prefix,$beam_width\ \omega$)
  for *c in char_set* $\mathcal{V}$ **do**
    $cur_prob \leftarrow LUT[cur_prefix] * next_prob[c]$
    if $cur_prob > \alpha$ **then**
      if $c == E_s$ **then**
        $\mathcal{P}.append(cur_prefix)$
      else
        $\mathcal{Q}.push(cur_prefix + c)$ /* The entire algorithm generates passwords by traversing each a trajectory τ that meets the criteria. */
        $LUT[cur_prefix + c] \leftarrow cur_prob$
return $\mathcal{P}$

Guesses generation. To align with the length distribution of actual passwords, we have developed an adaptive password generation algorithm. Initially, we start with an empty stack $\mathcal{Q}$ for prefixes and a lookup table LUT for storage as ASCII 32-126 we used can be found in Appendix A in Table VIII. The algorithm iterates by popping a prefix from the stack and predicting token probabilities with our guesser $\mathcal{G}_\theta$. If this probability exceeds the threshold or the token is E_s, then the prefix is a valid password and is added to the set $\mathcal{P}$. Otherwise, the prefix is extended with the token, its probability is updated in LUT, and it's pushed back onto the stack for popping. This recursive process continues until the stack is empty. Ultimately, the algorithm returns the set of generated passwords $\mathcal{P}$.

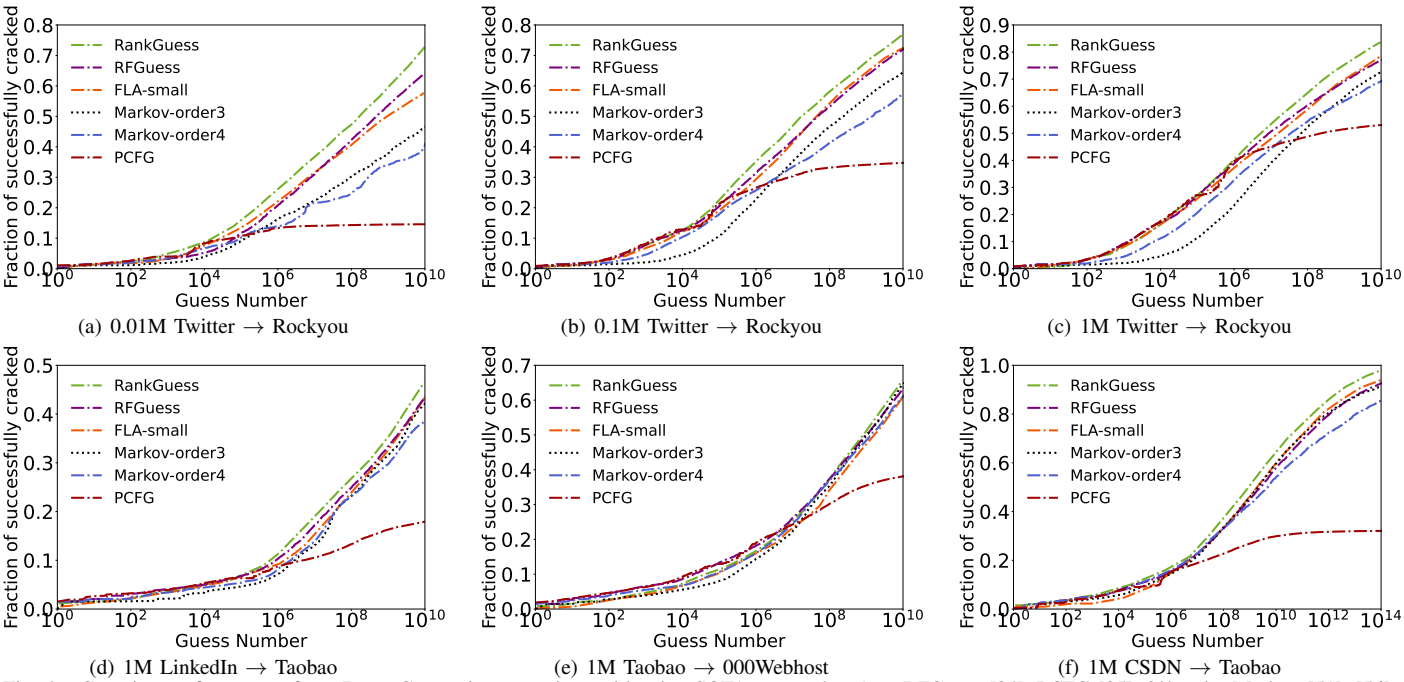
Hardware and software dependencies. The experiments were conducted on a workstation operating on Ubuntu 22.04 LTS. The models utilized were implemented using PyTorch[2] and TensorFlow[3] with their corresponding libraries. Notably, the experimental computing machine is equipped with an Intel Xeon Gold 6230R @2.1GHz CPU, 256G Memory, NVIDIA RTX 3090 GPU (with 24GB of VRAM), a 1TB solid-state drive and a 4TB hard disk drive. We posit that this configuration is attainable for real-world adversaries $\mathcal{A}$.

Datasets. We evaluate the existing password guessing approaches and our RANKGUESS model based on 12 large real-world password datasets (see Table II), a total of 421.79 million (M) passwords. Eight of our password datasets are from English sites and four are from Chinese ones. Our first step involves removing passwords consisting of symbols beyond the 95 printable ASCII tokens (*i.e.*, the Appendix. A in Table VIII). We have observed that the original datasets include abnormal passwords that do not seem to be user-chosen passwords or relevant information, either because they are too long (*exceeding* 30 tokens) or too short (*less than* 4 tokens) the same as previous work [53], [62], [84], [85]. Therefore, we have initiated the task of cleaning the datasets before conducting any experiments. These datasets exhibit variations in terms of web service, scale, time, with PII or not, password policy, and language, so it is sufficiently comprehensive to compare the performance of different SOTA models.

Ethical considerations. While the datasets in our research have been widely used in lots of literature [7], [60], [62], [81], [90], some of them remain private and contain sensitive information (see Table III). Our use of these datasets has received approval from our center's Institutional Review Board (IRB) and has been vetted by privacy experts. We take three precautionary measures to ensure that there is no additional harm to users: (1) All our datasets are stored and processed on computers not connected to the Internet; (2) To ensure the privacy of each individual account, we have only reported aggregated statistics of our research data; (3) Delete all the intermediate sensitive data (*e.g.*, email and targeted guesses) once our analysis is completed. Furthermore, our use of these datasets is not only beneficial for password guessing but also for admins to protect user account security, which is consistent with security research best practices (see, *e.g.*, [7], [62]).

Experimental setup. We compare RANKGUESS with leading guessing model (*i.e.*, PCFG [85], Markov [51], FLA [53], PassGAN [31] and RFGuess [84]). The setup and hyper-parameters of these compared models are as follows:

[2] https://pytorch.org/
[3] https://www.tensorflow.org/



(a) 0.01M Twitter → Rockyou (b) 0.1M Twitter → Rockyou (c) 1M Twitter → Rockyou

(d) 1M LinkedIn → Taobao (e) 1M Taobao → 000Webhost (f) 1M CSDN → Taobao

Fig. 4. Guessing performance of our RankGuess in comparison with other SOTA approaches (*i.e.*, RFGuess [84], PCFG [85], 3/4-order Markov [51], [56], FLA [53]) in the cross-site guessing scenarios with adaptive training sets and all fixed million(1M) training sets. → means adversaries use this portion of password information to attack users on another website. Our RankGuess performs better than its counterparts in most testing cases.

- **PCFG**. The PCFG we use is consistent with [51], that is, the probability of $\mathcal{L}$ segment comes from the trainset, which surpasses the original Weir *et al.* [85].
- **Markov**. For Markov, due to the influence of order, this work carries out both 3 and 4-order experiments simultaneously and adopts the Laplace smoothing and end symbol regularization as Ma *et al.* [51].
- **PassGAN**. The source code[1] of PassGAN [31] is not optimal, so we train it with 10,000 iteration and 10 gradient penalty. We sort the password candidates according to their frequency, merging the same passwords generated in the process.
- **FLA**. We use the source code[2] of FLA [53], and follow its recommended parameters in our experiments. More specifically, we train a model consisting of three LSTM layers with 200 cells (*i.e.*, *small* model in [53]) in each layer and two FC layers, 20 epochs.
- **RFGuess**. The RFGuess we use is consistent with Wang *et al.* [84], and we train the RF with 30 decision trees. Meanwhile, the minimum of leaf nodes is 10, and the maximum ratio of features is 80%.
- **RankGuess**. As detailed in the Algorithm 1, we train the model RankGuess with penalty $\gamma = 0.1$. The loss function utilized is KL divergence, and the optimizer is Adam [39], with an initial learning rate of 0.001 and betas of (0.5, 0.999) for momentum. We train the model with five pre-train epochs and a total of 20 epochs. More specifically, we train guesser $\mathcal{G}_\theta$ with 17 group numbers and 0.3 dropout rate. Meanwhile, we train ranker $\mathcal{R}_\varphi$ with a 0.3 dropout rate.

Experimental results. Given the computationally intensive nature of enumerating guesses, we employ the Monte-Carlo algorithm [18], [47] to approximate the number of attempts needed for adversaries $\mathcal{A}$ to discern a password through guesses made in decreasing order of likelihood. Our experiment is mainly divided into two parts: *(RQ1)* corroborating how much better RankGuess performs compared to other SOTA models and *(RQ2)* the impact of various factors on model performance: training size and language in Figure 4, 6 and 7. To answer the *RQ1*, we conduct an *apples-to-apples* performance comparison of these approaches using Twitter, LinkedIn, Taobao, CSDN to crack others (*i.e.*, Figure 4(c)∼4(f) and 7) Initially, the crack rates of all evaluated models are relatively low, making it difficult to discern any significant advantage. However, the superiority of the RankGuess begins to manifest distinctly at 10^4. In our cases, RankGuess outperformed SOTAs by 2.83%∼10.91%(avg.6.91%) and outperformed GAN-based SOTAs by 26.29%∼43.69%(avg.34.80%). The differences between different websites are significant and 1M trainset is

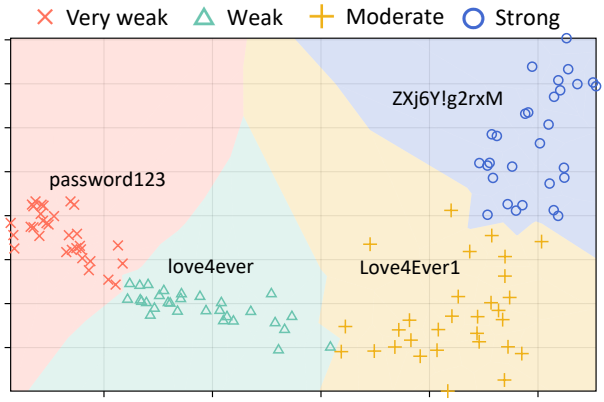


Fig. 5. The t-SNE-based visualization of ranker $\mathcal{R}_\theta$ latent space for passwords. We sample 10,000 passwords, projected them into Ranker's 512-dimensional feature space, then reduced them to 2D using t-SNE [76]. The visualization distinct clusters for very weak, weak, moderate, and strong passwords and red represents weaker passwords, such as `password123`, followed by green, yellow, and blue. The effectiveness of our Ranker model is consistent with the evaluation of the password strength meters as we expected.

[1] https://github.com/brannondorsey/PassGAN
[2] https://github.com/cupslab/neural_network_cracking

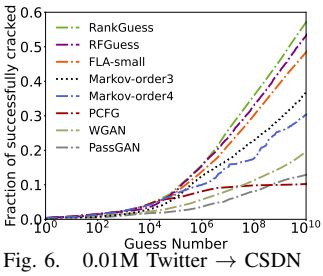


Fig. 6. 0.01M Twitter → CSDN

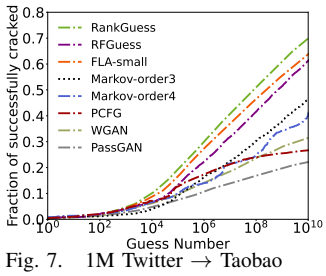


Fig. 7. 1M Twitter → Taobao

substantial in differentiating the performance.

Size and Language impact. To answer the *RQ2*, we train 0.01M∼0.1M Twitter to crack the same site (*i.e.*, Rockyou). We observed that all models improve with larger training sets. Notably, the PCFG [85] increased its performance from 12.75% to 53.90%, and RankGuess improved from 72.92% to 85.19%, particularly excelling with smaller training datasets. Analysis across 1M cases shows varying results; however, categorizing by language reveals distinct patterns. Models perform significantly better on English websites (Figures 4(c) and 4(e)) compared to Chinese sites (Figures 4(d), 4(f), and 7). We analyze the statistics for PW containing a *substring of 5+ letters* (Rockyou 71.69% and 000Webhost 67.82%), and the results are notably higher than for Chinese (CSDN 28.03%). This discrepancy explains why RankGuess performs better on English websites, as the sequence information is stronger. In addition, we visualized the latent space of the ranker $\mathcal{R}_\varphi$ for random sampled 0.1M passwords, reducing the dimensionality from 512 dimensions to 2 dimensions using t-SNE in Figure 5. The `password123` is the weakest, so it appears in the red. Next is `love4ever` in the green category. In the blue category are the strongest passwords (e.g., `ZXj6Y!g2rxM`). It is evident that ranker $\mathcal{R}_\theta$ exhibits a clear capability learning to rank which instructs the guessing process as Section I-A.

Further analysis. In our analysis of the most competitive models from the experiments, RFGuess [84] and FLA [53], we examined the passwords they successfully cracked. Overlap analysis, is commonly used in password research. The

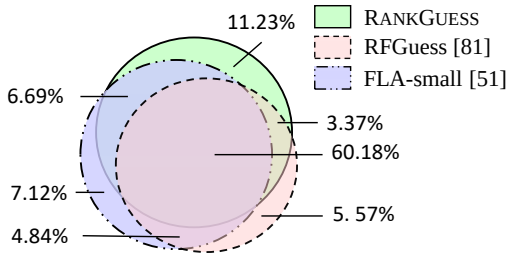


Fig. 8. The overlap ratios of cracked passwords by three models.

three models reveal that these models exhibit strong cracking capabilities, as they jointly cracked 60.18%. Furthermore, our analysis shows that the other two methods were able to generate 11.23% more unique passwords. This ability to produce a greater number of unique passwords constitutes the second source of improvement in our model's performance.

B. *Targeted Password Guessing* (Scenario#2)

There are many kinds of PII about users, such as demographic related information (name, birthday, age, occupation, education, gender, *etc.*). The goal of targeted password guessing is to crack the password of a given user in a given service as

TABLE III. Basic information of our PII datasets.

Dataset	Language	Items num	The types of PII attributes
PII-ClixSense	English	2,222,045	Email, User name, Name, Birthday
PII-000Webhost	English	79,580	Email, User name, Name, Birthday
PII-Rootkit	English	69,418	Email, User name, Name, Birthday
PII-12306	Chinese	129,303	Email, User name, Name, Birthday, Phone
PII-CSDN	Chinese	77,216	Email, User name, Name, Birthday, Phone
PII-Dodonew	Chinese	161,517	Email, User name, Name, Birthday, Phone

quickly as possible [81]. Targeted password guessing predicts the probability of the real passwords (PWs) given PII:

$$P(\text{PW} \mid \text{PII}) = \prod_{\substack{\text{PW}_i \in \text{PWs} \\ p_i \in \text{PII}}} P\left(\text{PW}_i \mid p_i, \mathcal{G}_\theta\right), \tag{12}$$

where PW denotes the password given personally identifiable information with guesser $\mathcal{G}_\theta$ and p_i are six attributes in PII.

Algorithm 3: Our PII matching algorithm.

Input: Passwords set $\mathcal{X} = \{pw_1, pw_2, ..., pw_n\}$, mapping from PII to labels $pattern2tags$. /* Pattern2tags are a series of pre-defined (personal information segment tag) tuples, indicating tagging to each PII.*/
Output: Passwords tagging PII structures $\mathcal{P}$.

```
for pattern, tag in pattern2tags do
    pos = P.match(pattern);/* Perform string matching to ascertain the presence of PII. */
    if pos! = −1 then
        for i = 0 to pattern.length() do
            for index = i + pos do
                tag_pwd = P.replace(index, tag)
            tag_pwds = PII_match(tag_pwd, pattern2tags);
        /* Conducting DFS recursively, with tag_pwds storing the matched password strings. */
        for i = 0 to pattern.length() do
            tag_pwd = pwd.replace(pos + i, pattern[i]);
for tag_pwd in tag_pwds do
    P.append(unique_tag(tag_pwd));/* Represent each PII with a single tag, eliminating unnecessary tags. */
return P
```

PII matching algorithm. For passwords which contain PII [81], such as `daniel1027` (*i.e.*, $\mathbf{N}_3$: `daniel` in name `danielsmith`, $\mathbf{B}_4$: `1027` in birthday `10271985`, *etc.*), we have noticed that such passwords can be split into PII segments, such as $\mathbf{N}_3\mathbf{B}_2$. Li *et al.* [44] introduced an approach for PII matching akin to the PCFG method [36], where, for instance, $\mathbf{N}_4$ denotes a name with a length of four, such as `wang`. It's evident that when PII is already known, splitting the password into segments based on letters as the basic tokens may lead to a loss of entropy of PII [84]. Using PII segments as the basis for dividing the password can better preserve semantics as proofed in [84]. For the password `wang1027`, compared to splitting it into token levels, tagging [6], [46], [50] PII can help avoid the loss of both the name and birthday information. Therefore, we need an effective PII matching algorithm for deep generative models to find PII segments.

Inspired by Wang *et al.*'s work [81], [84], the first procedure in our PII matching involves designing a set of tagging rules and establishing a series of tokenized labels to represent personal information. The summarization of these labels is

TABLE IV. COMPARISON OF FOUR PII-BASED MODELS.[†]

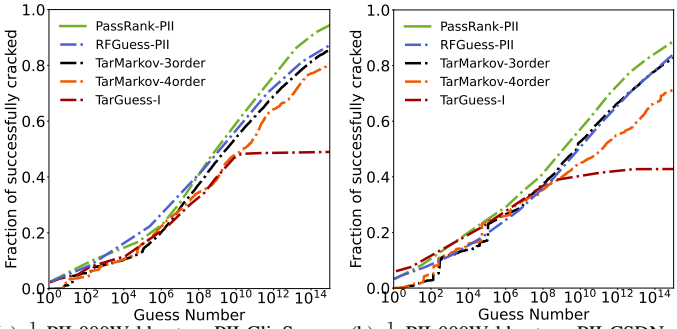
Experimental setup		RANKGUESS-PII	RFGuess-PII [84]	TarMarkov 3-order [79]	Tar-Guess-I [81]
Guessing scenario	Guess #				
50% PII-12306 ↓ 50% PII-CSDN	10	15.91%	**16.06%**	15.67%	8.41%
	10^2	**21.37%**	20.40%	20.09%	18.47%
	10^5	28.27%	27.20%	26.35%	**29.01%**
	10^8	**44.90%**	41.00%	43.27%	39.51%
	10^{15}	**70.45%**	66.58%	65.94%	56.05%
50% PII-CSDN ↓ 50% PII-ClixSense	10	4.15%	**5.20%**	4.12%	4.93%
	10^2	6.81%	**7.90%**	7.28%	5.96%
	10^5	**12.40%**	10.96%	11.45%	9.97%
	10^8	**27.83%**	25.46%	23.23%	24.85%
	10^{15}	**94.69%**	90.74%	85.17%	64.51%
50% PII-Dodonew ↓ 50% PII-000Webhost	10	2.51%	**3.07%**	2.40%	2.52%
	10^2	7.33%	**8.00%**	7.10%	6.18%
	10^5	**19.05%**	17.29%	16.50%	14.39%
	10^8	**24.84%**	22.62%	23.45%	21.72%
	10^{15}	**72.22%**	69.61%	68.86%	53.80%
50% PII-ClixSense ↓ 50% PII-12306	10	**14.82%**	11.87%	10.90%	9.12%
	10^2	**20.05%**	18.05%	17.70%	16.67%
	10^5	**32.36%**	30.06%	28.70%	29.15%
	10^8	**57.11%**	54.01%	51.48%	55.75%
	10^{15}	**99.29%**	97.33%	92.38%	78.60%
50% PII-Rootkit ↓ 50% PII-Dodonew	10	**6.96%**	5.95%	6.32%	5.47%
	10^2	**11.40%**	10.07%	10.46%	9.21%
	10^5	**16.17%**	14.88%	15.12%	12.25%
	10^7	**39.45%**	38.81%	27.73%	36.77%
	10^{14}	**89.81%**	86.10%	78.24%	56.91%
50% PII-000Webhost ↓ 50% PII-Rootkit	10	**10.82%**	8.75%	7.90%	6.76%
	10^2	**15.19%**	13.89%	14.10%	12.64%
	10^5	**27.82%**	25.17%	22.32%	21.71%
	10^8	**37.57%**	32.26%	30.34%	31.73%
	10^{15}	**93.93%**	90.45%	85.43%	60.64%

[†] A value in **bold** (attack success rate) indicates that it is the best within each respective row.

provided in Table IX. Our tagging rules start from 1000 as $\mathbf{N}$, where 1000 represents the complete PII, 1007 denotes the uppercase surname $\mathbf{N}_7$, 1004 represents the lowercase surname $\mathbf{N}_3$, 2000 indicates the complete birthday as $\mathbf{B}$, and so on. We have constructed a comprehensive and full tag-matching representation, illustrated in Figure 14. Based on these rules, a password containing personal information can be represented using a series of these labels as the new training trajectory.

The second procedure involves enumerating all possible outcomes for each password by substituting PII labels. For instance, consider the targeted password `daniel1027`. It can be represented in different ways: $\{\mathbf{N}_3\,\mathbf{B}_4\}$, $\{\mathbf{N}_3\,1, 0, 2, 7\}$, $\{\mathbf{N}_3\,\mathbf{B}_{10}, 2, 7\}$,$\{\mathbf{N}_3\,1, 0, \mathbf{B}_{11}\}$,$\{\mathbf{N}_3\,\mathbf{B}_{10}\,\mathbf{B}_{11}\}$,$\{\mathbf{E}_3\,\mathbf{B}_4\}$...,and $\{\mathrm{d}, \mathrm{a}, \mathrm{n}, \mathrm{i}, \mathrm{e}, \mathrm{l}, 1, 0, 2, 7\}$, a total of 17 representation methods. In contrast to Wang *et al.*'s works [81], [84], where a single representation is selected as the definitive one for each password, our generative PII-tagged approaches involve inputting potential representations into the model during the training process. Our approach deviates from conventional machine learning paradigms, as deep learning necessitates substantial training datasets and places less emphasis on intricate feature engineering and preprocessing techniques. In this context, multiple representations can serve as a part of data augmentation [71], [88]. This means that instead of using just one representation of the data, we utilize various possible representations of the same data. Our approach can help enhance the robustness and generalization capabilities of models. We summarize our PII matching in the Algorithm 3.

Datasets. We evaluate the existing guessing approaches and our RANKGUESS-PII model based on 6 large real-world datasets (see Table III). As indicated in Table II, there are three datasets originally containing diverse forms of PII. To facilitate comprehensive targeted guessing evaluations, we align the passwords that do not initially contain PII with these three PII-containing datasets through email matching. This results in a total of six datasets associated with the PII attribute, as



(a) $\frac{1}{2}$ PII-000Webhost → PII-ClixSense (b) $\frac{1}{2}$ PII-000Webhost → PII-CSDN

Fig. 9. Guessing performance of our RANKGUESS in comparison with other targeted approaches using personal information (*i.e.*, Targuess-I [81], 3/4-order TarMarkov [79], RFGuess-PII [84]) in the cross-site targeted guessing scenarios. Our RANKGUESS-PII demonstrates superior performance compared to our counterparts, especially at large guess number.

detailed in Table III with detailing language, and item count.

Approaches for comparison. The prevailing targeted guessing models that utilize personally identifiable information (PII) predominantly consist of TarGuess-I [81], which is rooted in PCFG [85], and TarMarkov [79], which is built upon the Markov model [51]. RFGuess-PII [84], on the other hand, is designed using a random forest classification approach based on *n*-grams. It's worth noting that the original TarMarkov model, as proposed by Wang *et al.* [79], primarily focuses on leveraging name information. However, it can be readily extended to include additional user details such as user name, birthday, email, and so forth in this paper to compare.

Experimental results. In this part, we also propose two research questions: *(RQ3)* How much better is RANKGUESS-PII compared to other SOTA models? and *(RQ4)* What is the impact of language, size, and PII on the users' password security? To accurately depict the success rates of various attack methods, we conduct 22 testing cases and present specific result values at certain guess numbers (*i.e.*, 10^5 and 10^8, as shown in Table IV and XIII). Traditional statistical methods, *i.e.*, Targuess-I, and traditional machine learning methods perform well with small guess numbers (*i.e.*, $\leq 10^3$). However, RANKGUESS-PII, which is based on DRL, outperforms them. It leads by 6.32% to 17.09%(avg. 12.70%). It is evident that RANKGUESS-PII is more stable and better suited for targeted password guessing using PII than RFGuess-PII [84], TarMarkov-3/4order [81], and Targuess-I [81].

Language, size, and PII impact. To answer *RQ4*, we also perform a *like-for-like* performance comparison of these approaches with regard to dataset size(Exp. #6–#8 in Table XIII) and language. It was also observed that for RANKGUESS-MASK, as the proportion of PII-Mixed training data increased from 50% to 90%, the effectiveness at 10^{14} guesses improved from 91.73% to 99.28%, representing an increase of 7.55%. We observe that the crack rate for Chinese is, avg., 9.83% higher than that for English, as shown in Table XIII #3A∼#3B and Table IV. It can be found that matched patterns of `Date_YYMMDD` and `11-digit phone numbers`, and found that the proportion of Chinese password sets containing `Date_YYMMDD` ranged from 16.67% to 27.30% (avg. 20.23%), which is significantly higher than the average of 2.01% for English. This indicates that Chinese users' passwords include a higher proportion of PII, making them more susceptible to targeted attacks. This is also a key reason for

TABLE V. PROVIDED ARE PIVOT SAMPLES AND THE CORESPONDING TOP-5 PASSWORDS CRACKED BY RANKGUESS-MASK IN ROCKYOU. THE MAIN RESULTS ARE CLASSIFIED ACCORDING TO PIVOT FREQUENCY. FOR INSTANCE, USING A PIVOT LIKE "**E**AET***" IN 10^7 GUESSING ATTEMPTS YIELD A SUCCESS RATE OF 95.20, IN WHICH "**ELEZABETH**", A WELL-KNOWN FEMALE FIRST NAME, EMERGES AS THE MOST PROBABLE (TOP-1) PASSWORD.

Pivot	***Common***			***Uncommon***			***Rare***			***Super-rare***		
Rank	****nnie*****	***ar*an*****	***u****123**	**l*ng***n***	***i**e*bo***	**e***a*et***	**1*3**67**0**	**my**ss**r***	**sli*****w**	***a*r*l*pp*r**	****o*eso*c*r**	**1*51*1*86****
1	minniewow	darkangel	august123	longhuong	fisherboy	elizabeth	1234567890	mypassword	sliveshow	hairclipper	ilovesoccor	10519148650
2	hunniebee	farhana11	bunnie123	longhorn1	singerboy	ericareta	1234567880	mypassw0rd	sliv0show		ilovesoccer	13512108611
3	hannie123	harkangel	hungry123	longhorns	silverboy	elixabeth	1234567650	mylesstorm	slideshow		llovesoccer	13510138639
4	jonniece1	maryangel	butter123	langhorn!	mickeyboy	elysabeth	1334567980	mypasswor1	slidwshow			
5	minnie123	marian123	turtle123	longhorn7	cinderbob	**elezabeth**	1234567000	mypasswor8	slinkylaw			
Crack (%)	**99.13**	**78.26**	**86.91**	**69.48**	**83.72**	**95.20**	**72.67**	**91.19**	**85.09**	**28.53**	**43.71**	**56.76**

the improved effectiveness of RANKGUESS-PII, as it more effectively identifies PII within users' passwords.

C. *Conditional Password Guessing* (Scenario#3)

Adversaries $\mathcal{A}$ have obtained an incomplete password of the victim by exploiting various side-channel attacks [62], [90] (shoulder surfing [19], keystroke audio feedback [21], [42], *etc.*), then initiates a conditional guessing attack. Conditional guessing models crack passwords given a partial one (*e.g.*, `e***a*et*`, `d****l*02*`). A simple way [62] to tackle this issue is by trawling models to create numerous potential passwords. However, the task of efficiently removing irrelevant passwords from this pool can be resource-intensive and demand significant storage [90]. In this work, the symbol $\star$ serves as a wildcard character that acts as a stand-in for an unspecified character. For example, the pivot `e***a*rt*` signifies a list of passwords beginning with `elizabeth` and followed by five or more unspecified characters.

Conditional password generation. Conditional guessing [62], [90] predicts the probability of the real passwords (PWs) conditioned on a pivot:

$$P(\text{PW} \mid pivot) = \prod_{\substack{\text{PW}_i \in \text{PWs} \\ p_i \in pivot}} P\left(\text{PW}_i \mid p_i, pivot, \mathcal{G}_\theta\right). \quad (13)$$

Finding an optimal guessing strategy that minimizes the expected guess rank is NP-hard. However, adversaries, $\mathcal{A}$ could still find approximate solutions that minimize the expected guess rank. We design a BFS with pruning as Algrithm 4 for conditional attacks. It generates a set of guesses $\mathcal{P}$, given a pre-collected password pivot, the number of guesses n, and the number of masked positions m. It employs an iterative process to systematically investigate potential characters for each wildcard position. This method involves choosing the most probable deep guessing candidates. The algorithm is not always optimal but still offers a fairly accurate approximation of the success rate of the optimal adversaries. Whether an efficient algorithm with tighter approximation bounds exists remains a challenging open research question.

Datasets. We evaluate the existing guessing approaches and our RANKGUESS-MASK model based on eight large real-world password datasets (see Table II) and combination. Five of our password datasets are from English sites and three are from Chinese ones. In strict adherence to the evaluation approach meticulously outlined in previous cutting-edge research [62], [90], we systematically employ randomly masked characters with a precisely calibrated 50% within a password. Furthermore, as a stringent criterion, we meticulously retain only those ingeniously generated pivots containing a minimum of five masked symbols and four discernible as [62], [90].

Evaluation. Our evaluation aligns with both RANKGUESS-MASK and existing methods like CWAE [62] and PassBERT [90]. We generated a substantial pool of 10^7 candidate passwords for each evaluation pivot. We classified these evaluation pivots into four categories based on the number of passwords satisfying each pivot, denoted as N_{pivots}: (1) ***common*** if $N_{\text{pivots}} \in [1000, 1500]$; (2) ***uncommon*** if $N_{\text{pivots}} \in [50, 150]$; (3) ***rare*** if $N_{\text{pivots}} \in [10, 15]$; (4) ***super-rare*** if $N_{\text{pivots}} \in [1, 5]$. These categories provide a nuanced assessment of pivot guessing performance across different pivot frequencies. Specific illustrative examples of evaluation pivots and the corresponding guesses and crack rate can be found in Table V.

Algorithm 4: Pruned BFS for conditional attack.

Input: Password pivot p, guess number n, masked number m.
Output: Passwords set $\mathcal{P}$.

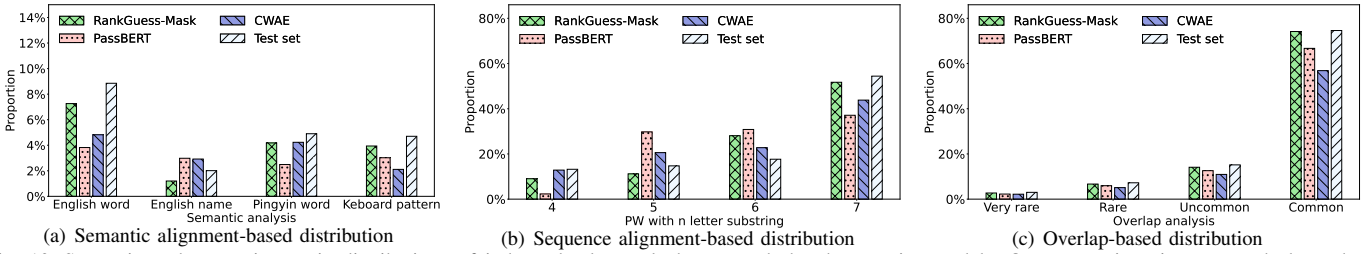
Search width $\omega = \lceil \sqrt[m]{n} \rceil$; /* Calculate the approximate possible states required for each position p. */
Empty queue $\mathcal{Q}$ with parallelizable pop operations;
Initialize the queue $\mathcal{Q}$, and $\mathcal{Q}.push((initial_state, prob = 1.0))$;
for masked position p_1 to p_n **do**
  while $!\mathcal{Q}.empty()$ **do**
    $cur_tuple = \mathcal{Q}.parallel_pop()$;
    for *cur_pivot, cur_pivot_prob* **in** *cur_tuple* **do**
      $next_probs = guesser\ \mathcal{G}_\theta(cur_pivots)$;
      Select top-ω $next_probs$ and $chars$ /* Select ω candidate characters for each p_i in the pivots template. */
      for *char,next_prob* **in** *chars,next_probs* **do**
        $next_pivot = cur_pivot.replace(p_i, char)$
        $new_prob = cur_pivot_prob * next_prob$
        $\mathcal{Q}.push(next_pivot, new_prob)$ /* Search for possible characters at each position p_i in the pivots template. */
Ouput password candidates $\mathcal{P} = \mathcal{Q}.parallel_pop()$
return $\mathcal{P}$

Experimental settings. We compare RANKGUESS-MASK with two SOTA models, *i.e.*, CWAE [62] and PassBERT [90]. The setups of RANKGUESS-MASK are identical to those outlined in Section IV, while the others are as follows:

CWAE. The CWAE we employed is in accordance with Pasquini *et al.* [62] and is implemented using the open-source[2] PasswordAE model, utilizing the default parameters.

PassBERT. For PassBERT [90] model, we use the source code[3] of Xu *et al.* [90], and follow its recommended parameters in our experiments. More specifically, to fine-tune on the pre-trained model released by PassBERT [90] (*i.e.*, pre-training on 60 M passwords sample from `Rockyou-2021`, which is a Rockyou breach happened in 2021), we train a model consisting of two hidden layers with each 128 hidden sizes, gelu hidden activation and Dropout rate 0.1.

[2] https://github.com/pasquini-dario/PLR
[3] https://github.com/snow0011/PassBertStrengthMeter



(a) Semantic alignment-based distribution (b) Sequence alignment-based distribution (c) Overlap-based distribution

Fig. 10. Semantic and syntactic metric distributions of independently cracked passwords by three major models. Our comparison is structured along three dimensions: semantic, sequence information, and overlap-based analysis. We systematically examine the advantages RankGuess-Mask gained across different models and test sets in all cases from #A1 to #C2 in Table VI. RankGuess-Mask excels at cracking passwords that consist of English and Pinyin words, as well as keyboard patterns and substring longer than 6 letters. It also achieves the best crack rate for *common* pivots compared to PassBERT [90] and CWAE [62].

Experimental results. To comprehensively assess our model's cracking ability and investigate its generalization capacity, we further design 16 attack cases. We divide the password sets into Chinese (CSDN, Dodonew, Taobao), English (Wishbone, 000Webhost, LinkedIn), mixed Chinese (*i.e.*, mix all Chinese passwords), mixed English password sets (mix all English passwords), and all mixed. Notably, in the Rockyou $\rightarrow$ Clixsense case at 10^7 guesses, it outperforms better than CWAE [62] by 14.85%, relevantly. Across all the cases, RankGuess-Mask shows an average success rate improvement of 8.21%, providing significant evidence of its superior cracking performance compared to PassBERT [90] and CWAE [62]. To evaluate the model's guessing performance at various guessing levels, we set three levels of guess numbers: low, moderate, and high(*i.e.*, 10, 10^4, 10^7). Based on the experimental results in Table X, RankGuess-Mask outperforms the SOTA level in all three guessing levels across the five cases.

TABLE VI. Comparison of conditional guessing models.†

Experimental setup		RankGuess-Mask	PassBERT [90]	CWAE [62]
Guessing scenario	Pivot			
CSDN → LinkedIn	*common*	85.57%	82.38%	95.44%
	uncommon	76.41%	72.86%	83.29%
	rare	66.50%	68.19%	84.84%
	super-rare	74.96%	60.29%	73.19%
CSDN → 000Webhost	*common*	79.14%	75.67%	76.87%
	uncommon	78.57%	74.26%	71.48%
	rare	40.02%	39.69%	38.94%
	super-rare	18.30%	24.81%	15.02%
Wishbone → Dodonew	*common*	88.21%	86.42%	79.43%
	uncommon	71.22%	70.29%	68.00%
	rare	56.36%	54.34%	49.73%
	super-rare	42.56%	48.06%	33.15%
Wishbone → Taobao	*common*	89.60%	87.52%	85.14%
	uncommon	84.03%	79.26%	80.76%
	rare	66.02%	67.08%	65.33%
	super-rare	55.42%	36.66%	39.28%
#A1: Engmix → Rockyou	*common*	99.41%	89.42%	76.20%
	uncommon	93.09%	83.10%	72.19%
	rare	92.30%	82.21%	70.35%
	super-rare	90.29%	74.81%	72.74%
#A2: Engmix → Taobao	*common*	89.14%	85.53%	88.15%
	uncommon	73.09%	70.26%	71.95%
	rare	75.86%	69.58%	71.90%
	super-rare	73.09%	70.59%	68.42%
#B1: Chmix → Rockyou	*common*	92.12%	91.80%	89.13%
	uncommon	89.52%	86.72%	83.19%
	rare	85.17%	81.16%	80.57%
	super-rare	83.46%	74.72%	50.19%
#B2: Chmix → Taobao	*common*	98.10%	96.72%	95.53%
	uncommon	93.47%	91.36%	90.65%
	rare	89.13%	87.30%	85.41%
	super-rare	84.19%	81.23%	80.58%
#C1: Mixed → LinkedIn	*common*	99.81%	97.29%	95.39%
	uncommon	94.01%	91.22%	90.94%
	rare	91.72%	87.32%	90.37%
	super-rare	89.85%	86.37%	85.68%
#C2: Mixed → Dodonew	*common*	98.59%	97.34%	94.71%
	uncommon	97.21%	95.99%	91.31%
	rare	91.35%	90.92%	88.87%
	super-rare	80.35%	60.41%	72.32%

† A value with dark gray (resp. light gray) represents the highest one (resp. 2nd one).

Language and PII impart. Moreover, we study the model's cracking performance when cracking passwords across languages (*i.e.*, Chinese $\rightarrow$ English, and vice versa) to investigate its generalization. In cases involving Chinese, English, and mixed training sets, RankGuess-Mask performs best in nearly all cases, improving performance by 7.09%. This shows the generalization ability of RankGuess-Mask. It is observed that RankGuess-Mask does not exhibit a clear preference for specific English or Chinese languages or PII as is previously noted in Sections IV and IV-C.

Case studies. A significant research question *(RQ5)* is identifying which types of passwords that RankGuess-Mask excels at cracking. To concretely answer it, we analyzed passwords generated by different models against our pre-collected semantic dictionaries, detailed in Appendix E (English, Name, Pinyin word, and Keyboard pattern). We discovered that RankGuess-Mask performs well in generating English words, Pinyin words, and keyboard patterns, but is less adept with names compared to PassBERT [90]. For passwords containing long substrings (more than five characters), RankGuess-Mask demonstrated improved results, aligning more closely with the ground truth distribution in the test dataset. Additionally, our findings show that passwords featuring common pivots were cracked more frequently, highlighting the effectiveness of our approach in differentiating between pivot categories. Through the analysis provided, the strengths of RankGuess-Mask have been identified.

V. Implications and Insights

A. Password strength meters

As one of the important applications of password cracking algorithms, Password strength meters (PSMs) are to accurately measure password strength from an attacker's perspective. PSMs [10], [16], [17], [23], [25], [61], [73], [78] show the strength of passwords to help users change a weak password to a strong one, since users' perception of password security is generally error-prone [78]. Labeling passwords as weak may cause usability problems while classifying them as strong could introduce security issues. Additionally, we have implemented a PSM. To assess the effectiveness of our meter, we gauge its performance using two accuracy metrics [26], [78] to detect unsafe errors [53] in passwords.

On the accuracy of PSMs. To obtain general results across meters in the real world, we refer to practices [26], [78] that count how meters' guesses for a given password are correlated with occurrence frequencies in an evaluation set using the weighted spearman coefficient (r_s) as Golla *et al.* [87]:

$$r_s = 1 - \frac{6 \sum d_i^2}{n\left(n^2 - 1\right)} \tag{14}$$

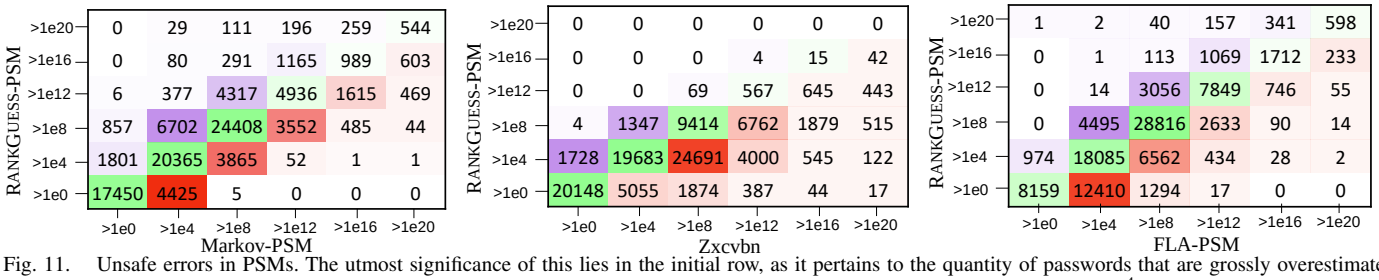


Fig. 11. Unsafe errors in PSMs. The utmost significance of this lies in the initial row, as it pertains to the quantity of passwords that are grossly overestimated by other meters. Our RankGuess meter assesses a total of 6,377 passwords as being guessable within the range of 1 to 10^4 guessing attempts, while the Zxcvbn [86] meter categorizes them as requiring more than 10^4 attempts. RankGuess-PSM and FLA-PSM [53] show similar performance on unsafe errors. The exaggerations of strength are visually and graphically represented in various shades of bold red, while underestimations are depicted in deep shades of purple, and precise evaluations are denoted in soothing shades of green. The color intensity escalates significantly in accordance with the number of passwords within a specific category just like the confusion matrix in machine learning, following the setting of precious works [26], [53], [89].

where $d_i = \text{rank}(\texttt{PSM}) - \text{rank}(\texttt{FRE})$, represents the difference between the ranks in PSMs and real frequency, and n is the number of passwords considered in the calculation.

TABLE VII. Accuracy (r_s) across PSMs, determined through the weighted Spearman correlation of five models with evaluation sets. A **bold** value signifies the best among them.

Dataset	***English***		***Chinese***	
PSMs	LinkedIn	Rockyou	CSDN	Dodonew
RankGuess	**0.652**	0.511	**0.728**	**0.535**
FLA-PSM [53]	0.367	0.355	0.643	0.322
Zxcvbn [86]	0.579	0.463	0.514	0.475
Markov-PSM [10]	0.201	0.337	0.268	0.297
Min_auto [74]	0.558	**0.694**	0.575	0.423

Each of these datasets is obtained by sampling 0.1M. As shown in Table VII, our meter achieves better accuracy than the Zxcvbn meter, and in half of the cases, it is better than Min_auto [74]. In evaluation sets using CSDN, RankGuess-PSM has a correlation of 0.728, compared with 0.643 and 0.514 in FLA-PSM [53] and Zxcvbn [86]. Among them, Markov-PSM performs the worst. Our RankGuess-PSM provides a more precise assessment of password strength according to the result shown in Table VII.

Unsafe errors. The unsafe errors [53] refer to rating easy-to-guess passwords as strong ones or namely, overestimation of password strength[4]. We investigate these meters' unsafe errors and show the average results using a 10w password randomly sampled from CSDN. Our RankGuess-PSM approach demonstrated superior accuracy, with up to 1.7 times fewer unsafe errors compared to Markov-PSM [10] and FLA [53] as PSM, as shown in Figure 11. We used our tuned RankGuess-PSM implementation described in Section IV. In addition, we find that, compared to others, the Zxcvbn meter's errors are often at very low guess numbers, which can be particularly unsafe. Overestimation of password strength can be disastrous since the seemingly secure passwords could be actually not safe or very vulnerable. For example, for the 10,000 most likely passwords, Zxcvbn [86] makes 183 utmostly unsafe errors. *Overall, our* RankGuess-PSM *has relatively few unsafe errors but is better accurate than other PSMs as mentioned in Figure 15 and Table VII.*

[4]We have obtained the approval from CMU for our research and usage of the Password Guessability Service (PGS), which access to get min_auto guess number of passwords using default configurations via https://pgs.ece.cmu.edu/.

B. Honeywords system

Personal honeyword. At CCS'13, Juels and Rivest [34] introduced a honeyword mechanism for password file compromise detection. This system generates $k-1$ honeywords per account (*e.g.*, $k = 20$ in [34]) and stores them alongside the actual password. When crafting honeywords for a user [34], the advanced method meticulously involves selecting password structures and substrings with similar frequency patterns to real passwords. The ultimate goal [5] is to meticulously create a set of honeywords that closely and convincingly mimic the characteristics of genuine passwords. As a leading password model, RankGuess-PII can generate honeywords for password leak detection. In this scenario, model size and generation speed aren't crucial. The server generates 20 to 40 honeywords per account, following [82]. If an attacker uses a honeyword, it triggers an alert, notifying the administrator of a potential password breach by potential cyber-attacks.

Conditional honeyword. The earlier honeywords system was capable of resisting trawling guessing and targeted attacks [5], [82] but was defenseless against conditional attacks. Conditional password guessing (this work and [62], [90]) is a crucial scenario where, for a user's password, we can employ a conditional attack to obscure a certain percentage (e.g., 40 $\sim$ 60% [62]) of randomly selected characters. Subsequently, using the conditional attack models, passwords with probabilities and user password probabilities equal to those of the masked characters can be generated. This can, to some extent, mitigate the security risks posed by such attacks, which are missings not considered in previous research.

VI. Conclusion

This paper, we introduce RankGuess, a novel password guessing framework that employs a learning-to-rank approach within a Markov Decision Process. This model simulates password generation as a sequential decision-making process, clarifying the goal of password guessing as learning to rank. Tested across 42 guessing cases, RankGuess outperforms existing models, showing a significant improvement in various guessing scenarios. Further, we propose a password strength meter to mitigate the risks of real-world guessing attacks. This work provides a brand new technical route for modeling users' password guessability and is an attempt from different perspectives leading towards general artificial intelligence.

REFERENCES

[1] *DIGITAL AROUND THE WORLD, 2023*. https://datareportal.com/global-digital-overview.

[2] *Yahoo's 2013 Email Hack Actually Compromised Three Billion Accounts*, 2013. https://www.wired.com/story/yahoo-breach-three-billion-accounts/.

[3] *Massive data leak exposes 700 million LinkedIn users' information*, Jun. 2021. https://www.bleepingcomputer.com/news/security/google-microsoft-can-get-your-passwords-via-web-browsers-spellcheck/.

[4] *Google, Microsoft can get your passwords via web browser's spellcheck*, Sep. 2022. https://www.bleepingcomputer.com/news/security/google-microsoft-can-get-your-passwords-via-web-browsers-spellcheck/.

[5] Akshima, D. Chang, A. Goel, S. Mishra, and S. K. Sanadhya. Generation of secure and reliable honeywords, preventing false detection. *IEEE Trans. Dependable Secur. Comput.*, 16(5):757–769, 2019.

[6] A. Amini, T. Liu, and R. Cotterell. Hexatagging: Projective dependency parsing as tagging. In *Proc. ACL 2023*, pages 1453–1464.

[7] J. Bonneau. *Guessing human-chosen secrets*. PhD thesis, University of Cambridge, UK, 2012.

[8] J. Bonneau, C. Herley, P. van Oorschot, and F. Stajano. Passwords and the evolution of imperfect authentication. *Commun. ACM*, pages 78–87, 2015.

[9] J. Bonneau, C. Herley, P. C. Van Oorschot, and F. Stajano. The request to replace passwords: A framework for comparative evaluation of web authentication schemes. In *Proc. IEEE S&P 2012*, pages 553–567.

[10] C. Castelluccia, M. Dürmuth, and D. Perito. Adaptive password-strength meters from Markov models. In *Proc. NDSS 2012*.

[11] H. S. Chang, M. C. Fu, J. Hu, and S. I. Marcus. An adaptive sampling algorithm for solving markov decision processes. *Operations Research*, 53:126–139, 2005.

[12] G. Chaslot, S. Bakkes, I. Szita, and P. Spronck. Monte-carlo tree search: A new framework for game ai. In *Proc. AAAI 2008*, pages 216–217.

[13] Z. Chen and X. Zhang. A reinforcement learning-based sequence generation algorithm for password guessing. In *GLOBECOM 2022*, pages 4891–4896, 2022.

[14] K. Cho, B. van Merrienboer, C. Gulcehre, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *Proc. EMNLP 2014*.

[15] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Proc. NIPS 2017*, pages 1–9.

[16] X. de Carné de Carnavalet and M. Mannan. From very weak to very strong: Analyzing password-strength meters. In *Proc. NDSS 2014*.

[17] X. de Carné de Carnavalet and M. Mannan. A large-scale evaluation of high-impact password strength meters. *ACM Trans. Inf. Syst. Secur.*, 18(1):1:1–1:32, 2015.

[18] M. Dell'Amico and M. Filippone. Monte carlo strength evaluation: Fast and reliable password checking. In *Proc. ACM CCS 2015*.

[19] G. Dessouky, T. Frassetto, and A. Sadeghi. Hybcache: Hybrid side-channel-resilient caches for trusted execution environments. In *Proc. USENIX SEC 2020*, pages 451–468.

[20] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proc.NAACL-HLT 2019*, pages 4171–4186.

[21] X. Dong, Z. Shen, J. Criswell, A. L. Cox, and S. Dwarkadas. Shielding software from privileged side-channel attacks. In *Proc. USENIX SEC 2018*, pages 1441–1458.

[22] K. Duh and K. Kirchhoff. Learning to rank with partially-labeled data. In *Proc. SIGIR 2008*, pages 251–258.

[23] S. Egelman, A. Sotirakopoulos, K. Beznosov, and C. Herley. Does my password go up to eleven?: The impact of password meters on password selection. In *Proc. ACM CHI 2013*, pages 2379–2388.

[24] L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization. In *Proc. ICML 2023*, pages 10835–10866, 2023.

[25] M. Golla and M. Dürmuth. On the accuracy of password strength meters. In *Proc. ACM CCS 2018*, pages 1567–1582.

[26] M. Golla, M. Wei, J. Hainline, L. Filipe, M. Dürmuth, E. Redmiles, and B. Ur. "what was that site doing with my facebook password?" designing password-reuse notifications. In *Proc. ACM CCS 2018*.

[27] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[28] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.

[29] D. B. Grimes, D. R. Rashid, and R. P. N. Rao. Learning nonparametric models for probabilistic imitation. In *Proc. NIPS 2006*, pages 521–528.

[30] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proc.CVPR2016*, pages 770–778.

[31] B. Hitaj, P. Gasti, G. Ateniese, and F. Perez-Cruz. Passgan: A deep learning approach for password guessing. In *Proc. ACNS 2019*.

[32] J. Ho and S. Ermon. Generative adversarial imitation learning. In *Proc. NIPS 2016*, pages 4565–4573.

[33] J. Ho and S. Ermon. Generative adversarial imitation learning. In *Proc. NIPS 2017*, pages 4565–4573, 2016.

[34] A. Juels and R. L. Rivest. Honeywords: Making password-cracking detectable. In *Proc. ACM CCS 2013*, pages 145–160.

[35] J. Justesen and T. Høholdt. Maxentropic markov chains. *IEEE Trans. Inf. Theory*, pages 665–667, 1984.

[36] J. Justesen and K. J. Larsen. On probabilistic context-free grammars that achieve capacity. *Inf. Control.*, pages 268–285, 1975.

[37] M. J. Keith, B. B. M. Shao, and P. J. Steinbart. The usability of passphrases for authentication: An empirical field study. *Int. J. Hum. Comput. Stud.*, pages 17–28, 2007.

[38] P. G. Kelley, S. Komanduri, M. L. Mazurek, R. Shay, T. Vidas, L. Bauer, N. Christin, L. F. Cranor, and J. Lopez. Guess again (and again and again): Measuring password strength by simulating password-cracking algorithms. In *Proc. IEEE S&P 2012*, pages 523–537.

[39] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proc. ICLR 2015*, pages 1–15.

[40] D. Kumor, J. Zhang, and E. Bareinboim. Sequential causal imitation learning with unobserved confounders. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, editors, *Proc. NIPS 2021*, pages 14669–14680.

[41] M. Kwon, S. M. Xie, K. Bullard, and D. Sadigh. Reward design with language models. In *Proc. NIPS 2023*.

[42] A. Kwong, W. Wang, J. Kim, J. Berger, D. Genkin, E. Ronen, H. Shacham, R. S. Wahby, and Y. Yarom. Checking passwords on leaky computers: A side channel analysis of chrome's password leak detect protocol. In *Proc. USENIX SEC 2023*, pages 7107–7124.

[43] S. Levine and P. Abbeel. Learning neural network policies with guided policy search under unknown dynamics. In *Proc. NIPS 2014*.

[44] Y. Li, H. Wang, and K. Sun. A study of personal information in human-chosen passwords and its security implications. In *Proc. IEEE INFOCOM 2016*, pages 1–9.

[45] K. Lin, D. Li, X. He, M. Sun, and Z. Zhang. Adversarial ranking for language generation. In *Proc. NIPS 2017*, pages 3155–3165.

[46] M. Lindemann, A. Koller, and I. Titov. Compositional generalization without trees using multiset tagging and latent permutations. In *Proc. ACL 2023*, pages 14488–14506.

[47] P. Liu, J. Blocki, and W. Bai. Confident monte carlo: Rigorous analysis of guessing curves for probabilistic password models. In *Proc. IEEE S&P 2023*, pages 626–644.

[48] T.-Y. Liu. Learning to rank for information retrieval. pages 225–331, 2009.

[49] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, and J. Tang. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proc. ACL 2022*, pages 61–68.

[50] L. Loukas, M. Fergadiotis, I. Chalkidis, E. Spyropoulou, P. Malakasiotis, I. Androutsopoulos, and G. Paliouras. Finer: Financial numeric entity recognition for XBRL tagging. In *Proc. ACL 2022*, pages 4419–4431.

[51] J. Ma, W. Yang, M. Luo, and N. Li. A study of probabilistic password models. In *Proc. IEEE S&P 2014*, pages 689–704.

[52] P. Mahmoudieh, D. Pathak, and T. Darrell. Zero-shot reward specification via grounded natural language. In *Proc. ICML 2022*, pages 14743–14752.

[53] W. Melicher, B. Ur, S. Komanduri, L. Bauer, N. Christin, and L. F. Cranor. Fast, lean and accurate: Modeling password guessability using neural networks. In *Proc. USENIX SEC 2016*, pages 175–191.

[54] N. Metropolis and S. Ulam. The monte carlo method. *Journal of the American statistical association*, 44:335–341, 1949.

[55] H. Mozaffari, V. Shejwalkar, and A. Houmansadr. Every vote counts: Ranking-based training of federated learning to resist poisoning attacks. In *Porc. USENIX SEC 2023*.

[56] A. Narayanan and V. Shmatikov. Fast dictionary attacks on passwords using time-space tradeoff. In *Proc. ACM CCS 2005*, pages 364–372.

[57] F. Neil. *List of Data Breaches and Cyber Attacks in 2023*, 2023. https://www.itgovernance.co.uk/blog/list-of-data-breaches-and-cyber-attacks-in-2023.

[58] M. Orsini, A. Raichuk, L. Hussenot, D. Vincent, R. Dadashi, S. Girgin, M. Geist, O. Bachem, O. Pietquin, and M. Andrychowicz. What matters for adversarial imitation learning? In *Proc. NIPS 2021*, pages 14656–14668.

[59] L. Page, S. Brin, R. Motwani, and T. Winograd. The pagerank citation ranking: Bringing order to the web. 1999.

[60] B. Pal, T. Daniel, R. Chatterjee, and T. Ristenpart. Beyond credential stuffing: Password similarity models using neural networks. In *Proc. IEEE S&P 2019*, pages 417–434.

[61] D. Pasquini, G. Ateniese, and M. Bernaschi. Interpretable probabilistic password strength meters via deep learning. In *Proc. ESORICS 2020*.

[62] D. Pasquini, A. Gangwal, G. Ateniese, M. Bernaschi, and M. Conti. Improving password guessing via representation learning. In *Proc. IEEE S&P 2021*, pages 265–282.

[63] J. Peters, K. Mulling, and Y. Altun. Relative entropy policy search. In *Proc. AAAI 2010*, volume 24, pages 1607–1612.

[64] M. L. Puterman. Markov decision processes. volume 2 of *Handbooks in Operations Research and Management Science*, pages 331–434. 1990.

[65] S. Sahin and F. Li. Don't forget the stuffing! revisiting the security impact of typo-tolerant password authentication. In *Proc. ACM CCS 2021*, pages 252–270.

[66] Y. Schroecker and C. L. I. Jr. State aware imitation learning. In *Proc. NIPS 2017*, pages 2911–2920.

[67] C. E. Shannon. A mathematical theory of communication. *Bell Syst. Tech. J.*, pages 379–423, 1948.

[68] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. pages 3008–3021.

[69] W. Sun, A. Venkatraman, G. J. Gordon, B. Boots, and J. A. Bagnell. Deeply aggrevated: Differentiable imitation learning for sequential prediction. In *Proc. ICML 2017*, pages 1–17.

[70] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.

[71] M. A. Tanner and W. H. Wong. The calculation of posterior distributions by data augmentation. *Journal of the American statistical Association*, 82(398):528–540, 1987.

[72] I. Tolstikhin, O. Bousquet, S. Gelly, and B. Schoelkopf. Wasserstein auto-encoders. In *Proc. ICLR 2018*, pages 1–16.

[73] B. Ur, P. G. Kelley, S. Komanduri, and et al. How does your password measure up? The effect of strength meters on password creation. In *Proc. USENIX SEC 2012*, pages 65–80.

[74] B. Ur, S. M. Segreti, L. Bauer, N. Christin, L. F. Cranor, S. Komanduri, D. Kurilova, M. L. Mazurek, W. Melicher, and R. Shay. Measuring real-world accuracies and biases in modeling password guessability. In *Proc. USENIX SEC 2015*, pages 463–481.

[75] H. Valizadegan, R. Jin, R. Zhang, and J. Mao. Learning to rank by optimizing ndcg measure. *Proc. NIPS 2009*.

[76] L. Van der Maaten and G. Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

[77] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Proc. NeurIPS 2017*, pages 5998–6008.

[78] D. Wang, X. Shan, Q. Dong, Y. Shen, and C. Jia. No single silver bullet: Measuring the accuracy of password strength meters. In *Proc. USENIX SEC 2023*, pages 947–964.

[79] D. Wang and P. Wang. The emperor's new password creation policies. In *Proc. ESORICS 2015*, pages 456–477.

[80] D. Wang, P. Wang, D. He, and Y. Tian. Birthday, name and bifacial-security: Understanding passwords of chinese web users. In *Proc. USENIX Sec 2019*, pages 1537–1555.

[81] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang. Targeted online password guessing: An underestimated threat. In *Proc. ACM CCS 2016*, pages 1242–1254.

[82] D. Wang, Y. Zou, Q. Dong, Y. Song, and X. Huang. How to attack and generate honeywords. In *Proc. IEEE S&P 2022*, pages 966–983.

[83] D. Wang, Y. Zou, Y. Xiao, S. Ma, and X. Chen. Pass2edit: A multi-step generative model for guessing edited passwords. In *Proc. USENIX Security 2023*, pages 983–1000.

[84] D. Wang, Y. Zou, Z. Zhang, and K. Xiu. Password guessing using random forest. In *Proc. USENIX SEC 2023*, pages 965–982.

[85] M. Weir, S. Aggarwal, B. de Medeiros, and B. Glodek. Password cracking using probabilistic context-free grammars. In *Proc. IEEE S&P 2009*, pages 391–405.

[86] D. Wheeler. zxcvbn: Low-budget password strength estimation. In *Proc. USENIX SEC 2016*, pages 157–173.

[87] C. Wissler. The spearman correlation formula. *Science*, 22(558):309–311, 1905.

[88] Q. Xie, Z. Dai, E. Hovy, T. Luong, and Q. Le. Unsupervised data augmentation for consistency training. *Proc. NIPS 2020*.

[89] M. Xu, C. Wang, J. Yu, J. Zhang, K. Zhang, and W. Han. Chunk-level password guessing: Towards modeling refined password composition representations. In *Proc. ACM CCS, 2021*, pages 5–20.

[90] M. Xu, J. Yu, X. Zhang, C. Wang, S. Zhang, H. Wu, and W. Han. Improving real-world password guessing attacks via bi-directional transformers. In *Proc. USENIX SEC 2023*, pages 1001–1018.

[91] L. Yu, W. Zhang, J. Wang, and Y. Yu. Seqgan: Sequence generative adversarial nets with policy gradient. In *Proc. AAAI 2017*.

[92] V. Zimmermann. *From the quest to replace passwords towards supporting secure and usable password creation*. PhD thesis, Technical University of Darmstadt, Germany, 2021.

APPENDIX

For readability, here we summarize the notations in Table IX. Then we provide a detailed derivation of the policy gradient, some discussion of alternative solutions, and supplementary experimental results in Section E.

A. Abbreviations and notations

TABLE VIII. TYPICAL CATEGORIES OF CHARACTERS IN THIS PAPER.

Character type	Symbols	Count
Lower letters	a b c d e f g h i j k l m n o p q r s t u w v x y z	26
Upper letters	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z	26
Digits	0 1 2 3 4 5 6 7 8 9	10
Special symbol	Space ! " # $ % & ' ( ) * + , – . / : ; < = > ? @ [\ ] ^ _ } { | } ~	33
Begin & End	B_s E_s	2
Total types	ASCII 32-126 and B_s E_s	97

Table VIII and IX summarizes the notations in this paper, as well as the various categories of characters and the begin and end symbols used in this paper, which are ASCII 32-126.

TABLE IX. THE ABBREVIATIONS AND NOTATIONS IN THIS PAPER.

Notations	Descriptions
B_s E_s	Begin symbol and End symbol.
PW $PW_{1:t}$	Password and partial password.
L_n D_n S_n	The letters, digits and special segment in length n.
$\mathcal{T}$ $\mathcal{E}$	Reward and expert.
$\mathbf{E}_1 \sim \mathbf{E}_3$	$\mathbf{E}_1$ means username/local part; $\mathbf{E}_2$ means the second-level domain; $\mathbf{E}_2$ means the top-level domain,$\cdots$.
$\mathbf{T}_1 \sim \mathbf{T}_3$	$\mathbf{T}_1$ means the whole phone number; $\mathbf{T}_2$ means three digits/area code; $\mathbf{T}_3$ means subscriber/line number,$\cdots$.
$\mathbf{N}_1 \sim \mathbf{N}_{10}$	$\mathbf{N}$ stands for name usages, $\mathbf{N}_1$ for the usage of full name, $\mathbf{N}_2$ for the abbr. of the full name (e.g., `ds` from `danielsmith`),$\cdots$.
$\mathbf{B}_1 \sim \mathbf{B}_{15}$	$\mathbf{B}$ stands for birthday usages, $\mathbf{B}_1$ for full birthday in MDY format (e.g., `10271985`), $\mathbf{B}_2$ for full birthday in YMD format, $\cdots$.
π (a, s) τ	π means password generation policy; (a, s) means guesser action and state pair; τ means a trajectory of password.

B. Proof of the policy gradient

We now prove our proposed policy gradient update algorithm (in Table 14). The goal of reinforcement learning [70] is to learn a policy $\pi_\theta(a_t|s_t)$ that maximizes the expected reward $\mathfrak{L}(\theta) = \mathbb{E}_{x_{1:T}\sim P_\theta(x_{1:T})}\left[\mathcal{T}\left(x_{1:T}\right)\right]$, as mentioned at Section I-A. One direct method is to search the policy space directly to find the optimal policy, known as policy search. Policy search [43], [63] is essentially an optimization problem and can be categorized into gradient-based optimization and non-gradient-based optimization. Policy search may not require value functions and can directly optimize the policy. The agent needs to explore parameterized policies that can handle continuous states and directly learn policies.

Policy gradient [70] is a gradient-based reinforcement learning method. Assuming that $\pi_\theta(a_t|s_t)$ is a continuously differentiable function with respect to θ, we can optimize the parameter θ using gradient ascent to maximize the objective function $\mathfrak{L}(\theta)$, of which the derivative of the objective function *w.r.t.* the policy parameters θ is given by:

$$\begin{aligned}\frac{\partial \mathfrak{L}(\theta)}{\partial \theta} &= \frac{\partial}{\partial \theta}\int P_\theta(\tau)\mathcal{T}(\tau)\mathrm{d}\tau \\ &= \int \left(\frac{\partial}{\partial \theta}P_\theta(\tau)\right)\mathcal{T}(\tau)\mathrm{d}\tau \\ &= \int P_\theta(\tau)\left(\frac{1}{P_\theta(\tau)}\frac{\partial}{\partial \theta}P_\theta(\tau)\right)\mathcal{T}(\tau)\mathrm{d}\tau \\ &= \int P_\theta(\tau)\left(\frac{\partial}{\partial \theta}\log P_\theta(\tau)\right)\mathcal{T}(\tau)\mathrm{d}\tau \\ &= \mathbb{E}_{\tau\sim P_\theta(\tau)}\left[\frac{\partial}{\partial \theta}\log P_\theta(\tau)\mathcal{T}(\tau)\right],\end{aligned} \tag{15}$$

where $\frac{\partial}{\partial \theta}\log P_\theta(\tau)$ represents the partial derivative of the function $\log P_\theta(\tau)$ with respect to θ. From Equation 15, it can be seen that the direction in which parameter θ is optimized is such that it increases the probability $P_\theta(\tau)$ of trajectories τ with higher total rewards $\mathcal{T}(\tau)$. Meanwhile, $\frac{\partial}{\partial \theta}\log P_\theta(\tau)$ could be partitioned into:

$$\begin{aligned}\frac{\partial}{\partial \theta}\log P_\theta(\tau) &= \frac{\partial}{\partial \theta}\log\left(P_\theta\left(s_0\right)\prod_{t=0}^{T-1}\pi_\theta\left(a_t \mid s_t\right)P_\theta\left(s_{t+1} \mid s_t, a_t\right)\right) \\ &= \frac{\partial}{\partial \theta}\left(\log P_\theta\left(s_0\right) + \sum_{t=0}^{T-1}\log \pi_\theta\left(a_t \mid s_t\right) + \sum_{t=0}^{T-1}\log P_\theta\left(s_{t+1} \mid s_t, a_t\right)\right) \\ &= \sum_{t=0}^{T-1}\frac{\partial}{\partial \theta}\log \pi_\theta\left(a_t \mid s_t\right).\end{aligned} \tag{16}$$

The derivative *w.r.t.* θ of $P_\theta\left(s_0\right)$ and $P_\theta\left(s_{t+1} \mid s_t, a_t\right)$ are equal to zero. Thus, it becomes evident that the gradient $\frac{\partial}{\partial \theta}\log P_\theta(\tau)$ is solely dependent on the policy function, and not on the state transition probabilities. As a result, the policy gradient, denoted as $\frac{\partial \mathfrak{L}(\theta)}{\partial \theta}$, can effectively be expressed using the following mathematical formulation:

$$\begin{aligned}\frac{\partial \mathfrak{L}(\theta)}{\partial \theta} &= \mathbb{E}_{\tau\sim P_\theta(\tau)}\left[\left(\sum_{t=0}^{T-1}\frac{\partial}{\partial \theta}\log \pi_\theta\left(a_t \mid s_t\right)\right)\mathcal{T}(\tau)\right] \\ &= \mathop{\mathbb{E}}_{\mathrm{PW}_{1:T}\sim \mathcal{G}_\theta}\left[\sum_{t=1}^{T}\sum_{a_t\in\mathbb{A}}\nabla_\theta \log \pi_\theta(a_t|\mathrm{PW}_{1:t-1})\delta_t\right].\end{aligned} \tag{17}$$

C. Proof of the MC rollout converge

Moreover, as the number of visits to each reward tends to infinity, each estimated reward by Monte Carlo rollout converges to $\pi_\theta\left(a_t \mid s_t\right)$. This is easily understood in the context of first-visit MC: each reward is an independently and identically distributed estimate with the finite variance of $\mathcal{Q}_{\theta,\varphi}(\mathrm{PW}_{1:t-1}, \mathcal{E})$, and by the law of large numbers, the sequence of these estimates' means converges to the expected value. Each mean itself is an unbiased estimate, and the standard error of its error decreases as $1/\sqrt{n}$, where n is the number of rewards used to compute the mean. Assuming we conduct N trials and obtain N trajectories, denoted

as $\tau^{(1)}, \tau^{(2)}, \ldots, \tau^{(N)}$, with corresponding total rewards of $\mathcal{Q}_{\theta,\varphi}(\tau^{(1)}), \mathcal{Q}_{\theta,\varphi}(\tau^{(2)}), \ldots, \mathcal{Q}_{\theta,\varphi}(\tau^{(N)})$, the total reward can be approximated as follows:

$$\lim_{N\to\infty} \mathbb{P}\left(\left|\frac{1}{N}\sum_{n=1}^{N} \mathcal{Q}_{\theta,\varphi}(\tau^{(n)}) - \mathcal{Q}_{\theta,\varphi}(\mathrm{PW}_{1:t-1}, \mathcal{E})\right| > \varepsilon\right) = 0. \tag{18}$$

D. Discussion of alternative solutions

Statistical approaches. Using PCFG [85], the complex structures in training datasets lead to low generation probabilities and an overestimation of security strength. PCFG [85] lacks an effective method to handle unseen basic structures, further complicating accurate generation as a larger guess number. The effectiveness of a Markov model in password cracking peaks at a fourth-order model, as higher orders lead to overfitting due to data sparseness and the dependency on the frequency of n+1-*gram* strings in the training dataset. To mitigate this issue, Ma *et al.* [56] recommend Laplace smoothing which adds a small constant to character frequencies post-training, yet this method mainly lessens rather than resolves the sparseness problem and overlooks crucial sub-string features. TarGuess-I [81] and TarMarkov [79] are variants of previous models.

Machine Learning approaches. RFGuess [84] constructs a sequence model using n-grams as classification features to predict the next tokens, while FLA utilizes RNNs for feature extraction using n-grams to predict the next ones. PassBERT [20] employs BERT as the base model for password guessing but faces shortcomings such as slow inference speed and its inadequacy for generation tasks. Conversely, CWAE [62] is limited to conditional password guessing as it optimizes the entire password as a whole, which complicates sequence generation and leverages additional advantages, like PII.

LLM-based approaches. We have observed that large language models (LLMs) excel in natural language generation tasks. However, our experiments suggest that this may not apply to password guessing. As discussed in Section II, passwords differ significantly from natural language; they are typically short and have a vocab of only 96 characters, compared to the 100K-prompt range found in natural language. Furthermore, while the goal of LLMs is to align with human preferences (RLHF), the objective of password guessing aligns with learning to rank, providing crucial insight for our research. Even RLHF, as employed by LLM, relies on RL techniques as ours (such as PPO [15], [24], [41], *etc.*) to boost performance. However, according to Occam's principle, we prioritize simplicity. Thus, we have opted for a solution with fewer parameters. Our main task is to solve a learning to rank *alignment* problem. Notably, a significant challenge with LLMs lies in their slow inference speed and difficulty in generating a large number of password guesses.

E. Supplementary experimental results

This comprehensive analysis offers valuable insights into password security techniques, significantly advancing our understanding and contributing to the enhancement of password security practices and robust cybersecurity measures. We considered 14 guess experiments of cross-site scenarios. A **bold** value in each row means that it is the highest one among all five methods in Table XIII. This section focuses on targeted guessing based on personal information (PII) in Table XIII, along with supplementary experiments on conditional password guessing in Table VI, to demonstrate the remarkable effectiveness of our model in those guessing scenarios.

TABLE X. COMPARISON OF CONDITIONAL PASSWORD GUESSING MODELS WITH PASSBERT [90] AND CWAE [62].†

Experimental setup		RankGuess-Mask	Pass-BERT [90]	CWAE [62]
Guessing scenario	Guess number			
CSDN → LinkedIn	10	28.12%	25.80%	29.13%
	10^4	59.80%	56.13%	61.19%
	10^7	75.86%	71.88%	85.19%
CSDN → 000Webhost	10	18.59%	16.82%	15.44%
	10^4	51.79%	49.86%	44.56%
	10^7	54.01%	53.61%	50.58%
Wishbone → Dodonew	10	24.14%	23.44%	19.19%
	10^4	56.00%	55.69%	44.34%
	10^7	67.65%	64.78%	57.58%
Wishbone → Taobao	10	30.17%	25.11%	27.70%
	10^4	60.29%	56.42%	52.14%
	10^7	73.77%	67.63%	64.37%
Rockyou → LinkedIn	10	23.12%	13.38%	16.06%
	10^4	61.77%	58.16%	44.01%
	10^7	78.52%	62.35%	72.69%
Rockyou → Clixsense	10	26.91%	25.12%	24.84%
	10^4	68.01%	58.69%	55.87%
	10^7	80.13%	66.99%	70.19%

† A value with dark gray (resp. light gray) represents the highest one (resp. 2nd one).

Comperasion with other RL methods. We also noticed that some precious works have also used RL techniques to trawling guessing, such as [13]. Our approach differs from simple RL as we address more of learning to rank alignment problem rather than directly optimizing for password guessing. Password guessing requires ranking the guess candidates, which has a clear preference, as described in Section I. Previous methods (*e.g.*, [13] did not address the core challenge of password guessing (*i.e.*, learning-to-rank) and merely solved learning-to-generate guessing tasks. Here, we provide a comparison:

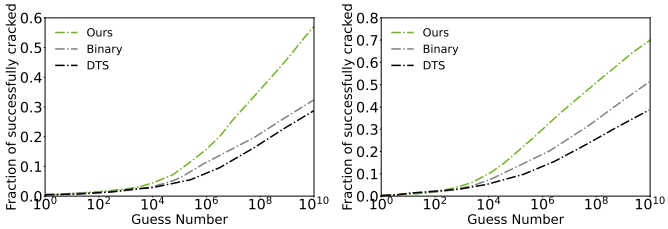


Fig. 12. 0.01M Twitter → CSDN Fig. 13. 1M Twitter → Taobao

Figure 12 and 13 show that learning to rank significantly enhances password guessing by optimizing in the direction of improvement, offering substantial gains over the traditional binary (true or false) approach denoted as Binary. Additionally, the Q-network described in [13] is utilized merely for Dynamic Temperature Sampling (DTS) and does not address the core issue of password guessing. Furthermore, their method is somewhat limited and cannot be applied to a broader range of password guessing scenarios in-the-wild.

Comperasion with GAN-based methods. As shown in Figure 6 and 7, GAN-based models perform less effectively than alternative existing models like PCFG [85], RFGuess [84], and FLA [53]. Particularly, PassGAN [31] needs ten times more guesses to get the same success rate as PCFG. Here we present comparison results under the same experimental setups: we compared our model with the previous GAN-based model *using a larger number of guesses*. We trained on 50% of them and then tested on Twitter. The results are as follows:

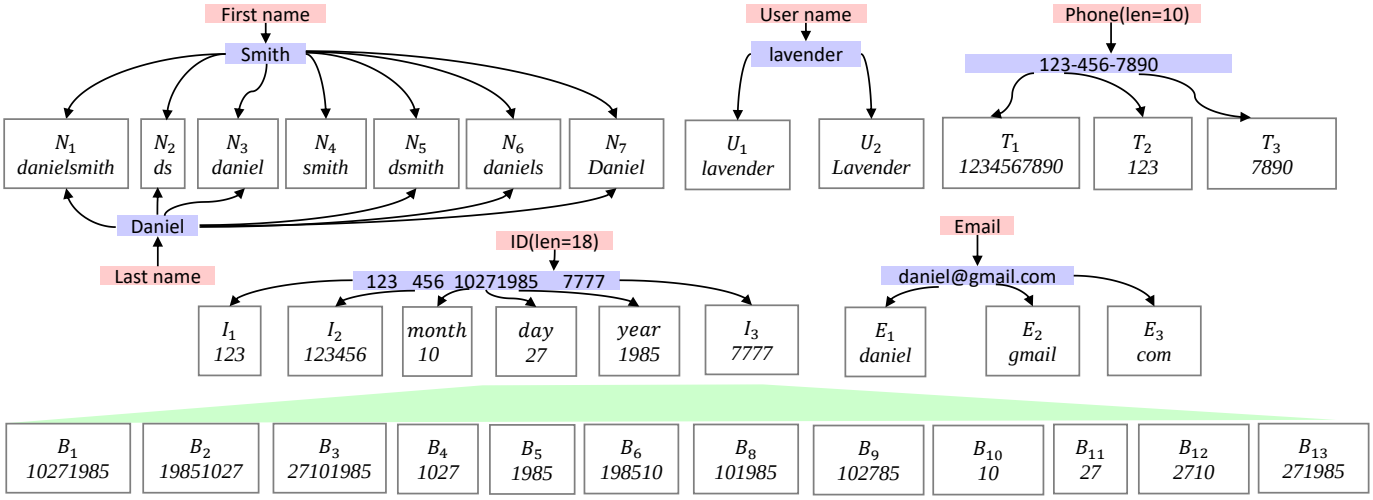


Fig. 14. An example of a process where a user's password is matched with their PII used by themselves is detailed. We have summarized the specific rules in Table IX. For instance, consider the targeted password `daniel1027`. It can be represented in different ways: $\{\mathbf{N}_3\,\mathbf{B}_4\}$, $\{\mathbf{N}_3\,1,0,2,7\}$, $\{\mathbf{N}_3\,\mathbf{B}_{10},2,7\}$,$\{\mathbf{N}_3\,1,0,\mathbf{B}_{11}\}$,$\{\mathbf{N}_3\,\mathbf{B}_{10}\,\mathbf{B}_{11}\}$,$\{\mathbf{E}_3\,\mathbf{B}_4\}\ldots$, and $\{\mathrm{d},\mathrm{a},\mathrm{n},\mathrm{i},\mathrm{e},\mathrm{l},1,0,2,7\}$, a total of 17 representation in our tagging methods.

TABLE XI. GUESSING PERFORMANCE OF OUR RANKGUESS IN COMPARISON WITH GAN-BASED MODELS (*i.e.*, PASSGAN [31] AND WGAN [62]) TRAINING ON 50% ROCKYOU DATASET TO CRACK THE TWITTER TEST SITE AT LARGE GUESS NUMBER.

Guess number	PassGAN [31]	WGAN [62]	RANKGUESS
$1\cdot10^8$	6.31%	8.67%	**52.12%**
$1\cdot10^9$	17.62%	22.53%	**60.51%**
$1\cdot10^{10}$	27.97%	35.49%	**70.67%**
$5\cdot10^{10}$	35.01%	46.37%	**73.34%**

Why do we choose cosine similarity? For Equation 5 using cosine similarity, when data differences are concentrated in a few dimensions, the *L1 norm* (Manhattan) can provide better performance. *L2 norm* (Euclidean) might require normalization when the scales of different dimensions vary. Cosine similarity is particularly well-suited for text because it ignores the impact of text length. It is mainly used in scenarios where measuring directional similarity, rather than magnitude, such as in passwords. We have verified this with experiments as Table XII. The performance of the Cosine method generally surpasses that of the Manhattan and Euclidean methods, with improvements ranging 8.34%∼28.19%. This indicates that under these specific test conditions, the Cosine method demonstrates superior password guessing performance, just as in [83], thus we adopted the Cosine for the experiments.

TABLE XII. THE GUESSING PERFORMANCE OF OUR RANKGUESS MODEL IS COMPARED USING COSINE, MANHATTAN, AND EUCLIDEAN SIMILARITY METRICS FOR TRAWLING, TARGETED, AND CONDITIONAL PASSWORD GUESSING SCENARIOS IN THIS PAPER.

Scenarios	**Cosine**	**Manhattan**	**Euclidean**
0.01M Twitter → CSDN	57.92%	49.58%	29.73%
50%PII-ClixSense→50%PII-12306	99.29%	89.01%	81.45%
Rockyou(Con.) → LinkedIn(Con.)	78.52%	64.95%	52.23%

Detailed information about our semantic dictionaries. To make our work as reproducible as possible and to facilitate the community, we now detail how to construct our semantic-based dictionaries in Figure 10. The first dictionary "English word" from the Oxford English Dictionary (OED) via https://www.oed.com/ includes 1M common lowercase English words. The second dictionary "English name" is a combination of the "firstname" and "lastname" featuring 1M commonly used English names from the American National Corpus (ANC) via the link https://anc.org/. The third dictionary pinyin words are from the Chinese Hanyu Pinyin Database https://www.cjk.org/data/chinese/nlp/chinese-hanyu-pinyin-database/. The last complex keyboard pattern is created by systematically identifying patterns in keyboard sequences that are more than three characters long and extend in various directions: *horizontal, vertical, diagonal, and their reverse*.

RANKGUESS-PSM for administrators. We have set up a toy usage example for website administrators or public safety maintainers, who can deploy our model as shown below to achieve the desired effect for users:

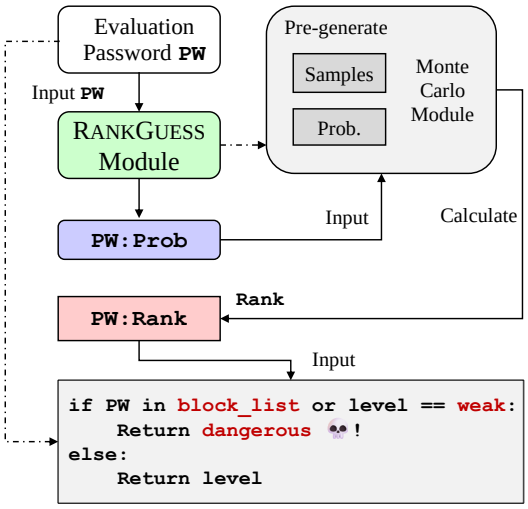


Fig. 15. Example for usage of RANKGUESS-PSM for strength evaluation.

Note that this usage method is not ***set in stone***. It can be customized to meet more advanced requirements for the website. The design of levels could be refined for finer decision-making and more detailed feedback [78]. For example, an additional feature could be added to provide suggestions for strengthening passwords based on certain characters to better guide users and add an extra module to provide PII checking in setting passwords that comply with the specific website policies.

TABLE XIII. COMPARISON OF THE CRACKING RATE OF FOUR DIFFERENT ATTACK METHODS OF CROSS-SITE SCENARIOS.

Experimental setup		RANKGUESS-PII	RFGuess-PII [84]	Targuess-I [81]	TarMarkov-3order [79]	TarMarkov-4order [79]
Attacking scenario	Number of guesses					
#1: 1M PII-000Webhost→PII-000Webhost_test	10	**8.36%**	5.78%	8.12%	1.95%	1.98%
	10^5	13.99%	**18.18%**	14.32%	13.08%	13.53%
	10^8	**23.38%**	23.22%	20.59%	22.16%	22.20%
	10^{14}	**73.06%**	62.52%	36.43%	51.46%	53.32%
#2: 1M PII-CSDN → PII-CSDN_test	10	14.36%	**24.20%**	14.55%	16.47%	15.55%
	10^5	**34.51%**	33.45%	34.11%	34.08%	34.48%
	10^8	**58.69%**	58.06%	34.08%	47.18%	48.92%
	10^{14}	**96.55%**	91.44%	43.67%	77.29%	79.99%
#3: 1M PII-12306 → PII-12306_test	10	12.53%	**16.32%**	11.28%	8.53%	8.54%
	10^5	**33.29%**	33.21%	29.88%	31.70%	31.56%
	10^8	57.92%	**59.97%**	34.11%	49.82%	46.07%
	10^{14}	**96.68%**	82.40%	44.86%	88.01%	83.43%
#4: 1M PII-Rootkit → PII-Rootkit_test	10	5.57%	**7.24%**	6.18%	4.91%	4.92%
	10^5	**33.22%**	23.21%	22.05%	20.21%	20.21%
	10^8	**42.78%**	42.54%	28.66%	29.98%	30.01%
	10^{14}	**89.37%**	82.40%	31.34%	69.89%	70.12%
#5: 1M PII-ClixSense → PII-ClixSense_test	10	8.36%	7.14%	**9.14%**	8.16%	8.11%
	10^5	**35.16%**	31.79%	27.55%	29.23%	30.25%
	10^8	**60.73%**	57.21%	31.63%	51.32%	52.43%
	10^{14}	**96.36%**	86.16%	56.93%	79.90%	84.21%
#6: PII-Mixed†: 90% → 10%	10	**18.07%**	15.36%	17.13%	16.97%	17.73%
	10^5	**31.91%**	29.50%	26.32%	27.89%	28.66%
	10^8	67.86%	**68.32%**	66.62%	63.69%	61.88%
	10^{14}	**99.28%**	96.36%	93.19%	91.82%	92.96%
#7: PII-Mixed: 70% → 30%	10	**16.55%**	15.36%	14.1%	13.94%	14.72%
	10^5	28.70%	27.60%	**29.31%**	26.63%	27.47%
	10^8	**62.67%**	60.17%	61.43%	57.58%	58.85%
	10^{14}	**93.60%**	91.25%	65.66%	88.23%	89.15%
#8: PII-Mixed: 50% → 50%	10	**16.17%**	15.16%	14.32%	12.86%	13.48%
	10^5	27.41%	26.78%	**28.76%**	26.90%	27.57%
	10^8	**57.16%**	56.44%	58.91%	49.77%	50.87%
	10^{14}	**91.73%**	89.23%	61.27%	85.50%	86.79%
#3A: $\frac{1}{2}$ PII-12306 → PII-CSDN	10	**11.88%**	11.28%	12.66%	10.28 %	10.30%
	10^5	**24.69%**	23.56%	22.13%	20.78%	21.53%
	10^8	**47.19%**	45.48%	46.16%	41.36%	40.19%
	10^{14}	**81.50%**	78.13%	62.64%	75.66%	76.96%
#3B: $\frac{1}{4}$ PII-12306 → PII-CSDN	10	**10.31%**	9.73%	9.87%	8.52%	7.61%
	10^5	**18.68%**	17.35%	18.39%	17.10%	15.39%
	10^8	**41.92%**	38.34%	40.32%	38.61%	36.53%
	10^{14}	**72.36%**	69.63%	58.98%	61.94%	60.73%
#3C: $\frac{1}{8}$ PII-12306 → PII-CSDN	10	**7.12%**	7.46%	7.65%	5.27%	6.71%
	10^5	**14.66%**	14.58%	13.79%	12.13%	12.78%
	10^8	**26.77%**	26.16%	24.11%	23.33%	22.66%
	10^{14}	**58.02%**	56.21%	49.66%	55.44%	51.14%
#3A: $\frac{1}{2}$ PII-ClixSense → PII-000Webhost	10	**9.47%**	9.13%	8.56%	8.18%	7.93%
	10^5	**18.01%**	17.49%	16.60%	16.30%	15.85%
	10^8	**23.89%**	23.12%	24.45%	22.40%	21.75%
	10^{14}	**62.50%**	58.88%	53.56%	55.05%	59.62%
#3B: $\frac{1}{4}$ PII-ClixSense → PII-000Webhost	10	8.51%	7.28%	**8.98%**	6.24%	7.18%
	10^5	**16.07%**	15.65%	15.39%	12.38%	14.90%
	10^8	**23.92%**	22.48%	22.59%	20.70%	23.07%
	10^{14}	**58.36%**	55.67%	52.09%	50.01%	51.48%
#3C: $\frac{1}{8}$ PII-ClixSense → PII-000Webhost	10	**8.48%**	7.39%	8.12%	8.01%	8.31%
	10^5	**14.66%**	13.76%	12.91%	13.20%	14.63%
	10^8	**25.21%**	24.94%	24.16%	21.03%	23.38%
	10^{14}	**57.02%**	55.20%	48.93%	52.23%	52.54%

† We mix three English datasets (*i.e.*, 000Webhost, Rootkit, and ClixSense) and three Chinese datasets (*i.e.*, CSDN, 12306, and Dodonew) and use 90%, 70% and 50% for training and the test 10%, 30% and 50% all of them. Further, elucidates the stability of our approach in the case of a larger training dataset.